
Diseño de Bases de Datos con Características de Orientación a Objetos



ASIGNATURA: BASES DE DATOS 2

PRÁCTICA 2

CURSO: 2025/26

COORDINADORA: Vázquez Martín, Lucía (NIP 871886)

Zhou Chen, Luna (NIP 812103)

Índice

Índice.....	2
1. Introducción.....	3
2. Diseño conceptual de la base de datos.....	3
2.1. Contexto del problema.....	3
2.2. Decisiones de diseño adoptadas.....	4
2.3. Restricciones del modelo.....	5
3. Diseño lógico relacional.....	7
3.1. Dominios de atributos y esquema relacional.....	7
3.2. Restricciones adicionales del esquema relacional.....	9
4. Implementación con modelo relacional (Oracle).....	10
4.1. Creación de las tablas.....	10
4.2. Triggers.....	12
4.3. Script de creación (bd-bancaria-oracle.sql).....	15
4.4. Pruebas relacionales (bd-bancaria-pruebas-oracle.sql).....	16
5. Diseño lógico objeto/relacional (SQL:1999).....	21
5.1. Definición de tipos objeto.....	21
5.2. Definición de tablas tipadas.....	23
5.3. Restricciones adicionales del modelo.....	25
6. Implementación con modelo objeto/relacional.....	26
6.1. Oracle (bd-bancaria-or-oracle.sql).....	26
6.2. PostgreSQL (bd-bancaria-or-postgresql.sql).....	28
6.3. DB2 (bd-bancaria-or-db2.sql).....	29
7. Generación de datos y pruebas.....	31
7.1. Oracle (bd-bancaria-or-pruebas-oracle.sql).....	32
7.2. PostgreSQL (bd-bancaria-or-pruebas-postgresql.sql).....	35
7.3. DB2 (bd-bancaria-or-pruebas-db2.sql).....	37
8. Implementación con db4o (Java).....	40
8.1. Compilación y ejecución.....	40
9. Comparación de SGBD.....	42
10. Dificultades y lecciones aprendidas.....	43
10.1. Diferencias entre SGBD en el soporte objeto-relacional.....	43
10.2. Gestión de dependencias entre objetos.....	44
10.3. Ejecución de scripts dentro de contenedores Docker.....	44
10.4. Limitaciones del cliente SQL*Plus y del uso del usuario SYS.....	44
10.5. Manejo de errores esperados en los scripts de pruebas.....	45
10.6. Reproducibilidad y automatización del proceso.....	45
11. Organización del trabajo y esfuerzos invertidos.....	46
12. Bibliografía.....	47
13. Anexos.....	48
13.1. Anexo I. Ejecución manual de los scripts en los distintos SGBD.....	48
13.2. Anexo II. Scripts SQL del modelo relacional.....	50
13.3. Anexo III. Scripts SQL del modelo objeto-relacional.....	57
13.4. Anexo IV. Implementación orientada a objetos (db4o).....	76

1. Introducción

La presente práctica tiene como objetivo el **diseño e implementación de una base de datos** para el dominio de una entidad bancaria, aplicando distintos modelos de datos y sistemas gestores. En primer lugar, se desarrolla el **diseño conceptual** mediante un diagrama E/R extendido, incorporando una jerarquía de generalización/especialización. A partir de dicho diseño, se obtiene el **modelo lógico relacional** y se implementa en Oracle, incluyendo scripts de creación y pruebas.

Posteriormente, se construye un **modelo objeto/relacional** conforme a SQL:1999 y se implementa en varios SGBD. Además, se realiza una **implementación orientada a objetos mediante db4o** desde Java. Finalmente, se comparan las soluciones obtenidas y se documentan las dificultades encontradas, con el fin de justificar las decisiones adoptadas y garantizar la reproducibilidad del trabajo.

2. Diseño conceptual de la base de datos

2.1. Contexto del problema

El problema planteado consiste en el diseño de una base de datos destinada a **gestionar información básica de una entidad bancaria**. En particular, se pretende almacenar información relativa a clientes, cuentas bancarias, oficinas y operaciones realizadas sobre dichas cuentas. Este escenario representa una **simplificación del funcionamiento real** de una entidad bancaria, pero permite ilustrar los distintos elementos necesarios para modelar el dominio mediante un esquema conceptual basado en el modelo E/R extendido.

En este contexto, **cada cliente** puede ser titular de una o varias cuentas bancarias, y una misma cuenta puede tener varios titulares. Las **cuentas registran información relevante** como su número de cuenta, el IBAN, la fecha de creación, el saldo actual y el historial de operaciones realizadas. Asimismo, se distinguen **dos tipos de cuentas: cuentas corrientes y cuentas de ahorro**. Las cuentas corrientes se encuentran asociadas a una oficina física del banco, mientras que las cuentas de ahorro se gestionan de forma electrónica y no están vinculadas a una oficina concreta.

Las **operaciones bancarias** contempladas incluyen **ingresos de dinero, retiradas de efectivo y transferencias entre cuentas de la misma entidad**. Cada operación se identifica mediante un código numérico y registra información como la fecha y hora de realización, la cantidad involucrada, la cuenta origen y, dependiendo del tipo de operación, la oficina donde se realiza o la cuenta destino en caso de transferencia.

Dado que el escenario descrito constituye una **simplificación**, durante el proceso de diseño se han adoptado una serie de **decisiones y supuestos** adicionales con el objetivo de acotar el problema y facilitar su modelado.

2.2. Decisiones de diseño adoptadas

En primer lugar, se ha considerado que el atributo **IBAN** identifica de forma única cada cuenta bancaria, mientras que el atributo **Número de cuenta** se mantiene como información adicional de interés para el sistema. Aunque ambos atributos están relacionados, el IBAN se utiliza como identificador principal debido a que constituye el estándar internacional utilizado para identificar cuentas bancarias en sistemas financieros.

El **número de cuenta** se modela como un **atributo de 20 dígitos**, correspondiente a la estructura tradicional utilizada en el sistema bancario español, formada por el código de entidad (4 dígitos), el código de oficina (4 dígitos), los dígitos de control (2 dígitos) y el número de cuenta propiamente dicho (10 dígitos). Mantener este atributo permite preservar información histórica y estructural relevante sobre la cuenta.

Respecto a la **relación entre clientes y cuentas**, inicialmente podría considerarse que cada cliente debe disponer obligatoriamente de al menos una cuenta. Sin embargo, desde el punto de vista del modelo de datos y su posterior implementación en los distintos SGBD, resulta más adecuado modelar esta relación con **cardinalidad (0,N)** desde cliente hacia cuenta. De este modo se evita imponer restricciones que podrían dificultar la creación inicial de los registros y se permite registrar clientes que todavía no han abierto ninguna cuenta.

En cuanto a la entidad **Oficina**, se ha considerado que el **atributo telefono** es un atributo simple, ya que se asume que cada oficina dispone de un único número principal de atención al cliente. De forma similar, en la entidad **Cliente** se ha modelado el **atributo telefono** como un atributo simple, suponiendo que cada cliente dispone de un único número de contacto registrado en el sistema, utilizado por la entidad bancaria para procesos de autenticación o validación de operaciones.

Asimismo, se asume que **cuando se crea una cuenta bancaria**, su saldo inicial es de 0 euros. Esta hipótesis simplifica el modelo y refleja el comportamiento habitual en la apertura de cuentas bancarias. De igual modo, se considera que actualmente las cuentas **no tienen comisiones de mantenimiento** y que no se permite la existencia de saldos negativos.

Las **cuentas corrientes se asocian a una oficina bancaria** concreta, ya que suelen abrirse de forma presencial en una sucursal. No obstante, se ha considerado que una oficina puede existir sin tener necesariamente cuentas asociadas en un momento determinado, por ejemplo en el caso de una oficina recién abierta. Por esta razón, la **relación entre oficina y cuentas corrientes** se modela con **cardinalidad (0,N)** desde oficina hacia cuenta.

En el caso de las **cuentas de ahorro**, se asume que proporcionan un determinado tipo de interés sobre el saldo depositado. Dicho interés se negocia con cada cliente y **se expresa como un porcentaje positivo**. Además, los intereses se devengan mensualmente, existiendo un procedimiento que se ejecuta periódicamente (por ejemplo, cada noche) para actualizar el saldo de aquellas cuentas de ahorro en las que corresponde aplicar intereses.

Finalmente, se considera que **tanto las cuentas corrientes como las cuentas de ahorro** pueden participar en operaciones bancarias, ya que ambas representan cuentas de depósito a la vista sobre las que se pueden realizar ingresos, retiradas y transferencias.

2.3. Restricciones del modelo

El sistema bancario modelado debe **cumplir una serie de restricciones de integridad** que garantizan la consistencia de los datos almacenados y reflejan determinadas reglas de negocio propias del dominio. Estas restricciones se agrupan en varias categorías.

1. Restricciones de herencia y tipado de entidades

R1. Toda cuenta registrada debe pertenecer obligatoriamente a exactamente uno de los subtipos definidos: CuentaAhorro o CuentaCorriente. No se permiten cuentas genéricas sin clasificar.

R2. Los subtipos CuentaAhorro y CuentaCorriente son disjuntos: una misma cuenta no puede pertenecer simultáneamente a ambos subtipos.

R3. Toda operación registrada debe pertenecer a uno de los tipos definidos en el sistema: Transferencia u Operación en efectivo (Ingreso o Retirada). No se permiten operaciones genéricas sin tipar.

R4. Las operaciones de tipo Transferencia deben incluir obligatoriamente una cuenta destino y no deben asociarse a ninguna oficina.

R5. Las operaciones de tipo Ingreso o Retirada deben estar asociadas a una oficina bancaria y no deben tener cuenta destino.

2. Restricciones temporales

R6. La fecha de creación de una cuenta no puede ser posterior al momento actual.

R7. La fecha y hora de cualquier operación debe ser anterior o igual al momento actual.

R8. La fecha de una operación debe ser mayor o igual que la fecha de creación de la cuenta origen.

R9. En el caso de una transferencia, la fecha de la operación también debe ser mayor o igual que la fecha de creación de la cuenta destino.

3. Restricciones de saldos

R10. Cuando se crea una cuenta bancaria, su saldo inicial debe ser 0 euros.

R11. El saldo actual de una cuenta debe ser siempre mayor o igual que cero.

R12. Solo se permite registrar una retirada o una transferencia si el saldo resultante de la cuenta origen sigue siendo mayor o igual que cero.

R13. Las cuentas de ahorro generan intereses positivos sobre el saldo depositado, calculados según el tipo de interés asociado a la cuenta.

R14. El saldo de las cuentas de ahorro se actualiza periódicamente mediante un procedimiento que aplica los intereses correspondientes.

4. Restricciones de operaciones bancarias

R15. En toda transferencia, la cuenta de origen y la cuenta de destino deben ser distintas.

5. Restricciones de integridad referencial

R16. Toda cuenta debe tener al menos un titular asociado.

R17. Toda operación debe estar asociada a una cuenta origen existente.

R18. Toda cuenta corriente debe estar asociada a una oficina bancaria existente.

6. Restricciones de dominio y formato de datos

R19. Todo cliente titular de cuentas debe tener edad mayor o igual que 18 años.

R20. Si se especifica una dirección de correo electrónico, esta debe seguir un formato válido (por ejemplo, contener el patrón **@*.**).

R21. El DNI de los clientes debe seguir el patrón 8 dígitos seguidos de una letra.

R22. El número de teléfono debe contener únicamente dígitos (habitualmente 9 dígitos con el patrón *[0-9]+*).

R23. El número de cuenta debe tener 20 dígitos, estructurados como: Entidad (4) + Oficina (4) + Dígitos de control (2) + Número de cuenta (10).

R24. En las cuentas de ahorro, el tipo de interés debe expresarse como un porcentaje positivo.

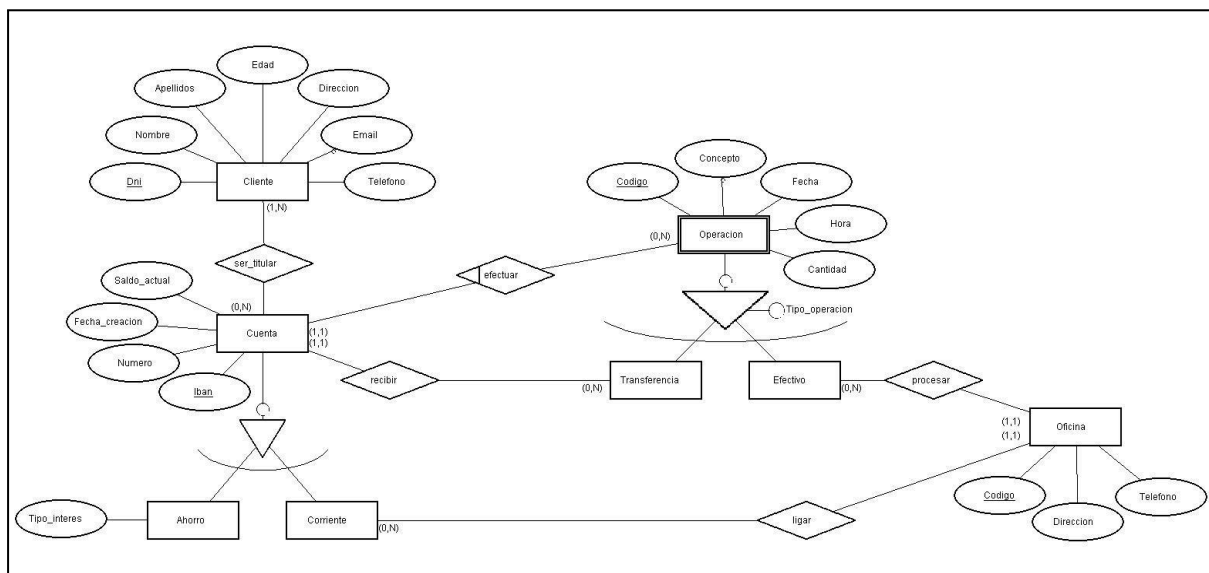


Figura 1. Diagrama conceptual para la gestión bancaria, representado mediante el modelo Entidad-Relación extendido.

3. Diseño lógico relacional

A partir del esquema conceptual definido mediante el modelo E/R extendido, se procede a realizar la **transformación al modelo lógico relacional**. Durante este proceso es necesario traducir las entidades, relaciones y jerarquías de especialización del modelo conceptual a un conjunto de relaciones (tablas) junto con sus correspondientes claves primarias, claves ajenas y restricciones de integridad.

En el esquema conceptual diseñado aparecen **dos jerarquías principales**: la jerarquía de **Cuenta** y la jerarquía de **Operacion**. Para su transformación al modelo relacional se han considerado distintas estrategias posibles, optándose en ambos casos por representar cada jerarquía mediante **una única tabla que agrupa la información del supertipo y de sus subtipos**.

La jerarquía formada por el **supertipo Cuenta** y sus **subtipos CuentaAhorro** y **CuentaCorriente** se ha transformado utilizando la estrategia de **una única tabla para toda la jerarquía**. De esta forma, la tabla resultante contiene tanto los atributos del supertipo como los atributos específicos de cada subtipo.

Esta decisión se ha adoptado por varias razones. En primer lugar, el **número de atributos específicos de los subtipos es reducido**, por lo que el número de valores nulos generados es limitado. Además, esta estrategia simplifica el diseño y evita la necesidad de realizar consultas que combinen varias tablas mediante operaciones de unión para reconstruir la información completa de una cuenta.

Para diferenciar entre ambos subtipos se utiliza el **atributo Es_ahorro**, que permite identificar si una cuenta corresponde a una cuenta de ahorro o a una cuenta corriente. En el caso de las cuentas de ahorro se almacena además el **atributo Tipo_interes**, mientras que en el caso de las cuentas corrientes se registra la oficina a la que se encuentra asociada.

De forma análoga, la jerarquía cuyo **supertipo es Operacion** se ha transformado también mediante **una única tabla que agrupa todos los tipos de operación**. En este caso, la decisión resulta adecuada porque el esquema conceptual ya incorpora un **atributo discriminador Tipo_operacion**, que identifica si una operación corresponde a un ingreso, una retirada o una transferencia. Este atributo permite distinguir entre los distintos tipos de operaciones sin necesidad de crear tablas adicionales para cada subtipo.

Los **subtipos de Operacion** únicamente introducen atributos adicionales relacionados con sus relaciones específicas: las operaciones en efectivo requieren una referencia a la **oficina** en la que se realizan, mientras que las transferencias requieren una referencia a la **cuenta destino**. Por tanto, el número de valores nulos en la tabla resultante es reducido.

3.1. Dominios de atributos y esquema relacional

Los **atributos del esquema relacional** se han definido utilizando dominios adecuados a la naturaleza de la información almacenada. En general se emplean **tipos de datos de carácter (VARCHAR)** para los atributos textuales, **tipos numéricos (ENTERO o REAL)** para valores numéricos y **tipos temporales (FECHA)** para representar información relacionada con fechas y horas. Además, se han incorporado **restricciones de nulidad (NO NULO)** en aquellos atributos cuyo valor es obligatorio según el modelo

conceptual, como el nombre y apellidos del cliente, la fecha de creación de la cuenta o la cantidad asociada a una operación.

A continuación se presentan las relaciones obtenidas tras la transformación del esquema conceptual al modelo relacional.

```
Cliente = (Dni: VARCHAR, clave primaria;
           Nombre: VARCHAR, NO NULO;
           Email: VARCHAR;
           Apellidos: VARCHAR, NO NULO;
           Edad: ENTERO, NO NULO;
           Direccion: VARCHAR, NO NULO;
           Telefono: ENTERO, NO NULO);

Oficina = (Codigo: ENTERO, clave primaria;
           Telefono: ENTERO, NO NULO;
           Direccion: VARCHAR, NO NULO);

Cuenta = (Iban: VARCHAR, clave primaria;
           Numero: ENTERO, NO NULO;
           Fecha_creacion: FECHA, NO NULO;
           Saldo_actual: REAL, NO NULO;
           Es_ahorro: ENTERO, NO NULO;
           Tipo_interes: REAL;
           Oficina_codigo: ENTERO);

           clave ajena (Oficina_codigo) referencia a Oficina(Codigo)

Operacion = (Codigo: ENTERO;
           Cuenta_origen: VARCHAR, NO NULO;
           Concepto: VARCHAR;
           Fecha: FECHA, NO NULO;
           Hora: HORA, NO NULO;
           Cantidad: REAL, NO NULO;
           Tipo_operacion: tpOperacion, NO NULO;
           Oficina_codigo: ENTERO;
           Cuenta_destino: VARCHAR);

           clave primaria (Codigo,Cuenta_origen);
           clave ajena (Cuenta_origen) referencia a Cuenta(Iban)
           clave ajena (Oficina_codigo) referencia a Oficina(Codigo)
           clave ajena (Cuenta_destino) referencia a Cuenta(Iban)

ser_titular = (Cliente_dni: VARCHAR;
           Cuenta_iban: VARCHAR);

           clave primaria (Cliente_dni,Cuenta_iban);
           clave ajena (Cliente_dni) referencia a Cliente(Dni)
           clave ajena (Cuenta_iban) referencia a Cuenta(Iban)
```


Esta última relación **ser_titular** permite representar la relación **muchos a muchos** existente entre clientes y cuentas, ya que un cliente puede ser titular de varias cuentas y una cuenta puede tener varios titulares.

3.2. Restricciones adicionales del esquema relacional

Durante la **transformación del modelo conceptual al modelo relacional**, algunas restricciones pueden representarse directamente mediante claves y relaciones. Sin embargo, otras restricciones requieren expresarse mediante condiciones adicionales sobre los valores de los atributos. En particular, la **representación de jerarquías mediante una única tabla** requiere introducir restricciones que garanticen la coherencia entre los atributos de los distintos subtipos.

R1. Para cada tupla de **Cuenta**, si los atributos correspondientes a un subtipo presentan valores, entonces los atributos propios de los demás subtipos deben ser nulos.

R2. Para cada tupla de **Cuenta**, al menos los atributos obligatorios de alguno de los subtipos deben ser no nulos, evitando tuplas que no representen efectivamente ni una cuenta de ahorro ni una cuenta corriente.

R3. Para cada tupla de **Operacion**, si los atributos correspondientes a un subtipo presentan valores, entonces los atributos propios de los demás subtipos deben ser nulos.

R4. Para cada tupla de **Operacion**, al menos los atributos obligatorios de alguno de los subtipos deben ser no nulos, evitando operaciones que no se correspondan con transferencia ni con operación en efectivo.

R5. Para toda ocurrencia de Dni en **Cliente**, debe existir al menos una ocurrencia asociada en la relación **ser_titular**, garantizando que todo cliente registrado es titular de, al menos, una cuenta.

R6. Para toda ocurrencia de Iban en **Cuenta**, debe existir al menos una ocurrencia asociada en la relación **ser_titular**, garantizando que toda cuenta registrada dispone de, al menos, un titular.

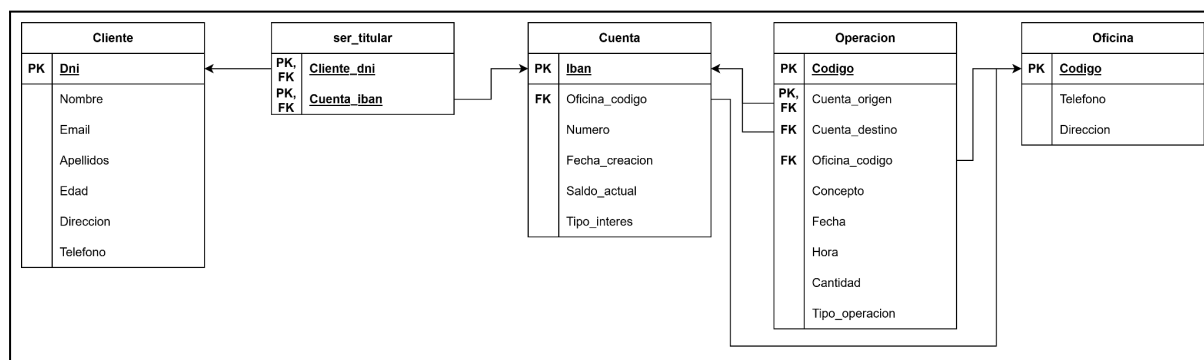


Figura 2. Diagrama lógico relacional para la gestión bancaria.

4. Implementación con modelo relacional (Oracle)

En este apartado se presenta la **implementación del esquema relacional** descrito en la sección anterior utilizando el sistema gestor de bases de datos (SGBD) Oracle. La creación de la base de datos se realiza mediante un **script SQL** denominado: **bd-bancaria-oracle.sql**

Este script contiene todas las sentencias necesarias para **crear las tablas, restricciones de integridad, triggers y procedimientos** requeridos por el sistema. Además, el script se ha diseñado de forma que pueda ejecutarse de manera independiente del estado previo de la base de datos, evitando conflictos con objetos previamente existentes.

A continuación, se presentan las sentencias utilizadas para la creación de las tablas.

4.1. Creación de las tablas

Las **tablas del esquema** se han definido en el script **siguiendo un orden que respeta las dependencias** entre ellas, con el fin de evitar errores derivados de las restricciones de integridad referencial. En primer lugar se crean las **tablas independientes** (Cliente y Oficina), seguidas de las **tablas que dependen** de ellas (Cuenta y Operacion) y, finalmente, la **tabla de relación ser_titular**, que referencia a ambas. Este orden permite ejecutar el script de creación sin necesidad de desactivar temporalmente las restricciones de clave ajena.

Tabla Cliente

La tabla Cliente almacena la información básica de los clientes de la entidad bancaria. En la **restricción que valida el formato del atributo Dni** se ha permitido que la letra final pueda introducirse tanto en mayúscula como en minúscula. Para ello se ha utilizado una expresión regular que acepta ambos casos ([a-zA-Z]). Esta decisión facilita la introducción de datos por parte de los usuarios sin afectar a la validez del identificador.

```
CREATE TABLE Cliente
(
  Dni          VARCHAR2(9) PRIMARY KEY,
  Nombre       VARCHAR2(50) NOT NULL,
  Email        VARCHAR2(50) CHECK (Email LIKE '%@%.%'),
  Apellidos    VARCHAR2(50) NOT NULL,
  Edad         NUMBER(3) NOT NULL CHECK (Edad >= 18),
  Direccion    VARCHAR2(100) NOT NULL,
  Telefono     NUMBER(9) NOT NULL,
  CHECK (REGEXP_LIKE(Dni, '^[0-9]{8}[a-zA-Z]$'))
);
```

Tabla Oficina

La tabla Oficina contiene la información relativa a las sucursales de la entidad bancaria.

```
CREATE TABLE Oficina
(
  Codigo       NUMBER(5) PRIMARY KEY,
  Telefono     NUMBER(9) NOT NULL,
```

```
Direccion    VARCHAR2(100) NOT NULL
);
```

Tabla Cuenta

La tabla Cuenta almacena tanto las cuentas corrientes como las cuentas de ahorro, siguiendo la estrategia de representación mediante una única tabla. La **restricción CHECK** garantiza la coherencia de la jerarquía: si una cuenta es de ahorro debe tener un tipo de interés asociado, mientras que si se trata de una cuenta corriente dicho atributo debe ser nulo.

```
CREATE TABLE Cuenta
(
    Iban            VARCHAR2(24) PRIMARY KEY,
    Numero          NUMBER(20) NOT NULL,
    Fecha_creacion  DATE DEFAULT SYSDATE NOT NULL,
    Saldo_actual    NUMBER(10,2) NOT NULL CHECK (Saldo_actual >= 0),
    Es_ahorro       NUMBER(1) NOT NULL CHECK (Es_ahorro IN (0,1)),
    Tipo_interes    NUMBER(5,2),
    Oficina_codigo  NUMBER(5),
    FOREIGN KEY (Oficina_codigo) REFERENCES Oficina(Codigo),
    -- Restricción de la jerarquía:
    --    si es ahorro (1) tiene interés y no tiene oficina,
    --    si es corriente (0) no tiene interés y sí tiene oficina.
    CHECK (
        (Es_ahorro = 1 AND Tipo_interes IS NOT NULL AND Oficina_codigo IS NULL)
        OR
        (Es_ahorro = 0 AND Tipo_interes IS NULL AND Oficina_codigo IS NOT NULL)
    );
```

Tabla Operacion

La tabla Operacion almacena todas las operaciones realizadas sobre las cuentas bancarias. En el modelo lógico relacional original se distinguían dos atributos temporales: Fecha y Hora. Sin embargo, en la implementación en Oracle se ha optado por utilizar un único **atributo denominado Fecha de tipo DATE**. Este tipo de dato en Oracle almacena tanto la fecha como la hora de forma conjunta, por lo que permite representar completamente la información temporal de la operación sin necesidad de definir un atributo adicional.

```
CREATE TABLE Operacion
(
    Codigo          NUMBER(8),
    Concepto        VARCHAR2(250),
    Fecha           DATE DEFAULT SYSDATE NOT NULL,
    Cantidad        NUMBER(10,2) NOT NULL CHECK (Cantidad > 0),
    Tipo_operacion  VARCHAR2(13) NOT NULL CHECK (Tipo_operacion IN ('Ingreso',
                                                                    'Retirada', 'Transferencia')),
    Cuenta_origen   VARCHAR2(24) NOT NULL,
    Oficina_codigo  NUMBER(5),
    Cuenta_destino  VARCHAR2(24),
```

```

PRIMARY KEY (Codigo, Cuenta_origen),
FOREIGN KEY (Cuenta_origen) REFERENCES Cuenta(Iban),
FOREIGN KEY (Oficina_codigo) REFERENCES Oficina(Codigo),
FOREIGN KEY (Cuenta_destino) REFERENCES Cuenta(Iban),

-- Restricciones de la jerarquía y lógica de negocio
-- Una cuenta no se transfiere a sí misma
CHECK (Cuenta_origen != Cuenta_destino),
CHECK ((Tipo_operacion = 'Transferencia' AND
        Cuenta_destino IS NOT NULL AND Oficina_codigo IS NULL) OR
        (Tipo_operacion IN ('Ingreso', 'Retirada') AND
        Oficina_codigo IS NOT NULL AND Cuenta_destino IS NULL) OR
        -- Excepción para los intereses del sistema:
        (Tipo_operacion = 'Ingreso' AND Concepto = 'Liquidacion de intereses
mensuales' AND Oficina_codigo IS NULL AND Cuenta_destino IS NULL))
);

```

Tabla ser_titular

La relación ser_titular permite representar la relación muchos a muchos existente entre clientes y cuentas.

```

CREATE TABLE ser_titular
(
    Cliente_dni    VARCHAR2(9),
    Cuenta_iban    VARCHAR2(24),
    PRIMARY KEY (Cliente_dni, Cuenta_iban),
    FOREIGN KEY (Cliente_dni) REFERENCES Cliente(Dni),
    FOREIGN KEY (Cuenta_iban) REFERENCES Cuenta(Iban)
);

```

4.2. Triggers

Algunas reglas del sistema no pueden expresarse mediante restricciones CHECK o claves ajenas, por lo que se implementan mediante triggers.

Trigger 1: Validación de la fecha de las operaciones

El siguiente trigger garantiza que la fecha de una operación no sea anterior a la fecha de creación de la cuenta origen ni, en el caso de transferencias, de la cuenta destino.

```

CREATE OR REPLACE TRIGGER trg_valida_fecha_operacion
BEFORE INSERT OR UPDATE ON Operacion
FOR EACH ROW
DECLARE
    v_fecha_creacion_origen DATE;
    v_fecha_creacion_destino DATE;
BEGIN
    -- Obtenemos la fecha de creación de la cuenta origen
    SELECT Fecha_creacion INTO v_fecha_creacion_origen

```

```

FROM Cuenta WHERE Iban = :NEW.Cuenta_origen;

IF :NEW.Fecha < v_fecha_creacion_origen THEN
    RAISE_APPLICATION_ERROR(-20001, 'Error: La operación es anterior a la
                                     creación de la cuenta origen.');
```

END IF;

-- Si es transferencia, comprobamos también la cuenta destino

```

IF :NEW.Tipo_operacion = 'Transferencia' AND :NEW.Cuenta_destino IS NOT NULL
THEN
    SELECT Fecha_creacion INTO v_fecha_creacion_destino
    FROM Cuenta WHERE Iban = :NEW.Cuenta_destino;

    IF :NEW.Fecha < v_fecha_creacion_destino THEN
        RAISE_APPLICATION_ERROR(-20002, 'Error: La operación es anterior a
                                         la creación de la cuenta destino.');
```

END IF;

END IF;

END;

/

Trigger 2: Actualización automática del saldo

Este trigger actualiza automáticamente el saldo de las cuentas cuando se registra una operación. En este caso, la verificación de que el saldo no se vuelva negativo se realiza mediante la restricción CHECK (Saldo_actual >= 0) definida en la tabla Cuenta, que impide que se registren operaciones que provoquen un saldo negativo.

```

CREATE OR REPLACE TRIGGER trg_actualiza_saldo
AFTER INSERT ON Operacion
FOR EACH ROW
BEGIN
    IF :NEW.Tipo_operacion = 'Ingreso' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;

    ELSIF :NEW.Tipo_operacion = 'Retirada' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;

    ELSIF :NEW.Tipo_operacion = 'Transferencia' THEN
        -- Restamos a la origen
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;
        -- Sumamos a la destino
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_destino;
    END IF;
END;
/
```

Trigger 3: Cálculo periódico de intereses

Para las cuentas de ahorro se ha implementado un procedimiento que calcula periódicamente los intereses generados. En primer lugar se define una secuencia para generar automáticamente los códigos de operación. A continuación se define el procedimiento encargado de calcular los intereses mensuales y registrar una operación de ingreso en la cuenta correspondiente. Finalmente, se programa una tarea automática mediante el planificador de Oracle (DBMS_SCHEDULER) que ejecuta dicho procedimiento diariamente durante la madrugada.

```
-- 1. Secuencia para generar los códigos de operación automáticamente
CREATE SEQUENCE seq_operacion START WITH 1 INCREMENT BY 1;

-- 2. Procedimiento para liquidar intereses
CREATE OR REPLACE PROCEDURE liquidar_intereses AS
    v_interes_calculado NUMBER(10,2);
BEGIN
    -- Recorremos todas las cuentas que sean de ahorro (1)
    FOR cuenta_rec IN (SELECT Iban, Saldo_actual, Tipo_interes, Oficina_codigo
                        FROM Cuenta
                        WHERE Es_ahorro = 1 AND Tipo_interes IS NOT NULL
                        AND Tipo_interes > 0)

    LOOP
        -- Calculamos el interés (p. ej. dividiendo tipo anual entre 12 meses)
        v_interes_calculado := cuenta_rec.Saldo_actual *
                               (cuenta_rec.Tipo_interes / 100 / 12);

        IF v_interes_calculado > 0 THEN
            -- Insertamos la operación como un 'Ingreso' por intereses
            -- Usamos la secuencia para el Código de operación
            INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad,
                                   Tipo_operacion, Cuenta_origen, Oficina_codigo, Cuenta_destino)
            VALUES (seq_operacion.NEXTVAL, 'Liquidación de intereses mensuales',
                    SYSDATE, v_interes_calculado, 'Ingreso', cuenta_rec.Iban,
                    cuenta_rec.Oficina_codigo, NULL);
        END IF;
    END LOOP;
    COMMIT;
END;
/

-- 3. Programación del JOB para que se ejecute de noche: a las 2 AM cada día
BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_LIQUIDAR_INTERESES',
        job_type           => 'PLSQL_BLOCK',
        job_action          => 'BEGIN liquidar_intereses; END;',
        start_date          => SYSTIMESTAMP,
        repeat_interval     => 'FREQ=DAILY; BYHOUR=2; BYMINUTE=0; BYSECOND=0',
        enabled             => TRUE,
```

```
comments          => 'Job para calcular y pagar intereses de cuentas de
                        ahorro cada madrugada'
);
END;
/
```

Cuando el **procedimiento liquidar_intereses** inserta una nueva **operación de tipo Ingreso** correspondiente a la liquidación de intereses, no es necesario actualizar explícitamente el saldo de la cuenta. Esto se debe a que el sistema ya dispone del **trigger trg_actualiza_saldo**, definido previamente sobre la **tabla Operacion**, que actualiza automáticamente el saldo de las cuentas cada vez que se registra una nueva operación. De esta forma se evita duplicar la lógica de actualización del saldo y se mantiene una única regla centralizada para gestionar los cambios en el balance de las cuentas.

4.3. Script de creación (bd-bancaria-oracle.sql)

La **creación del esquema relacional en Oracle** se realiza mediante el script `bd-bancaria-oracle.sql`, cuyo contenido completo se incluye en el **anexo correspondiente** de esta memoria. En dicho script se recogen todas las sentencias necesarias para construir la base de datos, descritas en los subapartados 4.1 y 4.2.

Para ejecutar el script en el entorno de prácticas, primero **se copia el archivo al interior del contenedor Docker** que ejecuta Oracle, concretamente al directorio temporal `/tmp`, mediante el siguiente comando:

```
root@core ~/bd2/sql/oracle# docker cp bd-bancaria-oracle.sql oracle-xe:/tmp/
Successfully copied 8.19kB to oracle-xe:/tmp/
```

Una vez copiado el fichero, el **script se ejecuta desde el propio contenedor** utilizando la **herramienta SQL*Plus**, que permite interpretar y ejecutar sentencias SQL y PL/SQL sobre la base de datos Oracle:

```
root@core ~/bd2/sql/oracle# docker exec -it oracle-xe sqlplus system/oracle123
@/tmp/bd-bancaria-oracle.sql
```

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mon Mar 9 18:04:59 2026
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Last Successful login time: Mon Mar 09 2026 18:03:46 +00:00
```

```
Connected to:
```

```
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

```
PL/SQL procedure successfully completed.
```

```
PL/SQL procedure successfully completed.  
PL/SQL procedure successfully completed.  
PL/SQL procedure successfully completed.  
PL/SQL procedure successfully completed.  
PL/SQL procedure successfully completed.  
PL/SQL procedure successfully completed.  
PL/SQL procedure successfully completed.  
  
Table created.  
Table created.  
Table created.  
Table created.  
Table created.  
  
Trigger created.  
Trigger created.  
  
Sequence created.  
  
Procedure created.  
  
PL/SQL procedure successfully completed.  
  
SQL>
```

Durante la ejecución se muestran distintos mensajes indicando la creación de los objetos definidos en el script (tablas, triggers, secuencias y procedimientos), lo que permite comprobar que la estructura de la base de datos se ha generado correctamente.

4.4. Pruebas relacionales (bd-bancaria-pruebas-oracle.sql)

Una vez creado el esquema de la base de datos, se procede a verificar su correcto funcionamiento mediante un **segundo script** denominado bd-bancaria-pruebas-oracle.sql, cuyo contenido completo se incluye también en el **anexo correspondiente** de esta memoria. **Este script contiene un conjunto de inserciones, consultas y operaciones** diseñadas para comprobar tanto el funcionamiento básico del sistema como el cumplimiento de las restricciones de integridad definidas en el modelo.

El **procedimiento de ejecución del script** es análogo al utilizado para el script de creación. En primer lugar, el archivo se copia al directorio temporal del contenedor Docker que ejecuta Oracle y posteriormente se ejecuta mediante la herramienta SQL*Plus sobre la base de datos creada previamente.

Las pruebas incluidas en el script se organizan en varios bloques con distintos objetivos:

1. Se realiza la **inserción de un conjunto de datos base** que permite inicializar el sistema con información válida sobre oficinas, clientes y cuentas. Esta fase permite comprobar que las tablas y las relaciones entre ellas han sido definidas correctamente y que el sistema admite datos consistentes con el modelo.
2. Se ejecutan diversas **pruebas de restricciones de integridad**, cuyo objetivo es verificar que el sistema detecta correctamente situaciones inválidas. Entre estas pruebas se incluyen intentos de **inserción de datos que incumplen restricciones** del modelo, como DNIs con formato incorrecto, clientes menores de edad, cuentas de ahorro sin tipo de interés o transferencias sin cuenta destino.
3. Se realizan **operaciones bancarias** válidas, como ingresos, retiradas y transferencias entre cuentas. Estas operaciones permiten **comprobar el funcionamiento de los triggers** definidos, en particular el encargado de actualizar automáticamente el saldo de las cuentas tras registrar una operación.
4. Se incluye una prueba destinada a **verificar el comportamiento del sistema ante situaciones de fondos insuficientes**. En este caso se intenta registrar una retirada superior al saldo disponible en la cuenta. El trigger actualiza inicialmente el saldo, pero la restricción CHECK definida en la tabla Cuenta impide la operación, produciendo un error y evitando que la cuenta quede con saldo negativo.
5. Se incluye una **prueba del procedimiento de cálculo de intereses** implementado para las cuentas de ahorro. Este procedimiento genera automáticamente una operación de ingreso correspondiente a la liquidación de intereses y permite comprobar que el saldo de la cuenta se actualiza correctamente como consecuencia de dicha operación.

En conjunto, las pruebas realizadas permiten verificar que el sistema implementa correctamente las restricciones del modelo relacional y que los mecanismos adicionales definidos mediante triggers y procedimientos funcionan según lo esperado.

```
root@core ~/bd2/sql/oracle# docker cp bd-bancaria-pruebas-oracle.sql
oracle-xe:/tmp/
Successfully copied 7.68kB to oracle-xe:/tmp/

root@core ~/bd2/sql/oracle# docker exec -it oracle-xe sqlplus system/oracle123
@/tmp/bd-bancaria-pruebas-oracle.sql

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Mar 9 18:07:05 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

Last Successful login time: Mon Mar 09 2026 18:04:59 +00:00

Connected to:

Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production

Version 21.3.0.0.0

Session altered.

=====

0. LIMPIEZA DE DATOS PREVIOS

=====

0 rows deleted.

0 rows deleted.

0 rows deleted.

0 rows deleted.

0 rows deleted.

Commit complete.

Datos anteriores eliminados correctamente.

=====

1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE)

=====

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

Commit complete.

Datos base insertados.

```
=====
2. PRUEBAS DE RESTRICCIONES (AQUI DEBEN SALIR ERRORES ORA-)
=====
> Prueba 2.1: Intentar insertar un DNI falso (Falla por Regex)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008321) violated

> Prueba 2.2: Intentar insertar un menor de edad (Falla por CHECK)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008320) violated

> Prueba 2.3: Cuenta de Ahorro sin tipo de interes (Falla por CHECK de
jerarquia)
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo,
Saldo_actual)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008332) violated

> Prueba 2.4: Transferencia sin Cuenta Destino (Falla por CHECK de operacion)
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion,
Cuenta_origen, Oficina_codigo, Cuenta_destino)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008342) violated

=====
3. PRUEBAS DE TRIGGERS Y OPERACIONES (DEBEN FUNCIONAR)
=====
> Hacemos un Ingreso de 1000 euros en la cuenta de Ana

1 row created.

> Hacemos una Retirada de 200 euros en la cuenta de Ana

1 row created.

> Ana hace una transferencia de 300 euros a Luis

1 row created.

> ESTADO DE SALDOS ESPERADO:
```

```

> Ana empezo con 0 + 1000 - 200 - 300 = 500 euros.
> Luis empezo con 0 + 300 transferidos = 300 euros.

IBAN                                SALDO_ACTUAL
-----
ES9100001111222233334444          500
ES9100005555666677778888          300

=====

4. PRUEBA DE DISPARADOR EN CADENA (FONDOS INSUFICIENTES)
=====

> Ana intenta sacar 600 euros pero solo tiene 500.
> El trigger restara el saldo y Oracle parara la insercion por el CHECK de saldo
>= 0.
VALUES (seq_operacion.NEXTVAL, 'Compra TV', 600, 'Retirada',
'ES9100001111222233334444', 1, NULL)

*
ERROR at line 2:
ORA-02290: check constraint (SYSTEM.SYS_C008330) violated
ORA-06512: at "SYSTEM.TRG_ACTUALIZA_SALDO", line 7
ORA-04088: error during execution of trigger 'SYSTEM.TRG_ACTUALIZA_SALDO'

> Comprobamos que el saldo de Ana sigue intacto en 500 euros

IBAN                                SALDO_ACTUAL
-----
ES9100001111222233334444          500

=====

5. PRUEBA DEL PROCEDIMIENTO DE INTERESES
=====

> Ejecutamos el pago de intereses manual (Liquidacion de la cuenta de ahorro de
Luis)

PL/SQL procedure successfully completed.

> Comprobamos las operaciones. Deberia aparecer un nuevo Ingreso automatico para
Luis

CODIGO CONCEPTO                                CANTIDAD TIPO_OPERACIO
-----
2 Nomina                                1000 Ingreso
3 Cajero                                200 Retirada
4 Regalo                                300 Transferencia
6 Liquidacion de intereses mensuales      .62 Ingreso

> Comprobamos que el saldo de Luis ha subido (Tenia 300, y se le ha sumado su
interes)

```

IBAN	SALDO_ACTUAL
-----	-----
ES9100005555666677778888	300.62

Commit complete.

PRUEBAS FINALIZADAS.

SQL>

5. Diseño lógico objeto/relacional (SQL:1999)

En esta fase se realiza el **diseño lógico de la base de datos** utilizando el **modelo objeto-relacional** definido en el **estándar SQL:1999**. Este modelo extiende el modelo relacional tradicional incorporando características propias de los sistemas orientados a objetos, como **tipos definidos por el usuario (UDT)**, **herencia entre tipos** y **referencias entre objetos (REF)**.

En este diseño, cada entidad principal del modelo conceptual se representa mediante un **tipo estructurado (CREATE TYPE)**. Estos tipos contienen los atributos propios de cada entidad y pueden incluir referencias hacia otros tipos. Posteriormente, estos tipos se materializan en la base de datos mediante **tablas tipadas**, definidas con la sentencia **CREATE TABLE OF**, lo que permite almacenar instancias de dichos tipos. Asimismo, algunas relaciones del modelo conceptual se representan mediante **colecciones de referencias (arrays de REF)**.

5.1. Definición de tipos objeto

En primer lugar se definen los **tipos base del sistema**, que representan las principales entidades del dominio: clientes, cuentas, operaciones y oficinas. Para evitar problemas de dependencia entre tipos que se referencian mutuamente, se declaran inicialmente algunos tipos de forma adelantada. Posteriormente se completan sus definiciones incluyendo los atributos y referencias correspondientes.

Los tipos **ClienteUdt** y **CuentaUdt** incorporan colecciones de referencias que permiten representar la relación de titularidad entre clientes y cuentas. De este modo, un cliente puede mantener una colección de cuentas asociadas y, de forma análoga, una cuenta puede contener una colección de referencias a sus titulares.

En el caso de las **cuentas bancarias** se define una jerarquía de tipos en la que **CuentaUdt** actúa como supertipo, mientras que **AhorroUdt** y **CorrienteUdt** representan los subtipos correspondientes a cuentas de ahorro y cuentas corrientes. Esta jerarquía se implementa mediante la cláusula **UNDER**, que permite heredar los atributos del tipo base.

De forma similar, las **operaciones bancarias** se modelan mediante el tipo base **OperacionUdt**, del cual derivan los tipos **TransferenciaUdt** y **EfectivoUdt**, que representan respectivamente las transferencias entre cuentas y las operaciones realizadas en efectivo en una oficina.

Un aspecto relevante del diseño es la definición de las acciones referenciales asociadas a las referencias (REF). En particular, para las referencias hacia cuentas en las operaciones se ha definido la acción **ON DELETE CASCADE**, ya que una operación pierde su significado si la cuenta asociada deja de existir. En cambio, para las referencias a oficinas en operaciones en efectivo se ha utilizado **ON DELETE SET NULL**. Esta decisión garantiza que el cierre de una sucursal no suponga la eliminación del historial de operaciones, sino que únicamente se desvincule la oficina de dichos registros.

```
CREATE TYPE CuentaUdt; -- Para evitar fallos de referencia cuando se crea el
ClienteUdt
CREATE TYPE OficinaUdt; -- Para evitar fallos de referencia cuando se crea el
EfectivoUdt

CREATE TYPE ClienteUdt AS (
    Dni          VARCHAR(9),
    Nombre       VARCHAR(50),
    Email        VARCHAR(50),
    Apellidos    VARCHAR(50),
    Edad         DECIMAL(3,0),
    Direccion    VARCHAR(100),
    Telefono     DECIMAL(9,0),
    refCuenta ref(CuentaUdt) array [10] scope Cuenta references are checked on
delete set null)
instantiable not final ref is system generated;

CREATE TYPE CuentaUdt AS (
    Iban         VARCHAR(24),
    Numero       DECIMAL(20,0),
    Fecha_creacion DATE,
    Saldo_actual DECIMAL(10,2),
    refCliente ref(ClienteUdt) array[5] scope Cliente references are checked on
delete set null)
instantiable not final ref is system generated;

CREATE TYPE AhorroUdt under CuentaUdt AS (
    Tipo_interes DECIMAL(10,2))
instantiable not final;

CREATE TYPE CorrienteUdt under CuentaUdt AS (
    refOficina ref(OficinaUdt) scope Oficina references are checked on delete set
null)
instantiable not final;

CREATE TYPE OperacionUdt AS (
    Codigo       DECIMAL(8,0),
    Concepto     VARCHAR(250),
    Fecha        TIMESTAMP,
    Cantidad     DECIMAL(10,2),
    refCuentaOrigen ref(CuentaUdt) scope Cuenta references are checked on delete
cascade)
```

```

instantiable not final ref is system generated;

CREATE TYPE TransferenciaUdt under OperacionUdt AS (
    refCuentaDestino ref(CuentaUdt) scope Cuenta references are checked on delete
    cascade)
instantiable not final;

CREATE TYPE EfectivoUdt under OperacionUdt AS (
    refOficina ref(OficinaUdt) scope Oficina references are checked on delete set
    null)
instantiable not final;

CREATE TYPE OficinaUdt AS (
    Codigo          DECIMAL(5,0),
    Telefono         DECIMAL(9,0),
    Direccion        VARCHAR(100),
    refCorriente     ref(CorrienteUdt) array[1000] scope CuentaCorriente references
    are checked on delete set null,
    refEfectivo      ref(EfectivoUdt) array[1000] scope OperacionEfectivo references
    are checked on delete set null)
instantiable not final ref is system generated;

```

5.2. Definición de tablas tipadas

Una vez definidos los tipos objeto, se procede a **crear las tablas tipadas** que almacenarán las instancias de dichos tipos. Estas tablas se crean mediante la sentencia **CREATE TABLE OF**, que indica que cada fila de la tabla corresponde a un objeto del tipo especificado.

En el diseño se distinguen dos grupos de tablas:

- **Tablas base**, que corresponden a los tipos principales del sistema (Oficina, Cliente, Cuenta y Operacion).
- **Tablas hijas**, que representan las especializaciones de los tipos base dentro de las jerarquías definidas.

En las tablas base se definen las claves primarias y las restricciones de integridad correspondientes a los atributos obligatorios. Además, se genera automáticamente un identificador de referencia (REF) para cada objeto almacenado en la tabla mediante la cláusula **REF IS ... SYSTEM GENERATED**.

Las tablas hijas heredan automáticamente los atributos y el identificador del tipo base, añadiendo únicamente las restricciones específicas de cada subtipo. Por ejemplo, en la tabla **CuentaAhorro** se incluye una restricción que obliga a que el tipo de interés esté definido y sea positivo, mientras que en **CuentaCorriente** se exige que exista una referencia válida a una oficina.

```

-- =====
-- 1. TABLAS BASE
-- =====

```

```

CREATE TABLE Oficina of OficinaUdt (
    PRIMARY KEY (Codigo),
    CONSTRAINT ofi_telefono CHECK (Telefono IS NOT NULL),
    CONSTRAINT ofi_direccion CHECK (Direccion IS NOT NULL),
    ref is id_oficina system generated
);

CREATE TABLE Cliente of ClienteUdt (
    PRIMARY KEY (Dni),
    CONSTRAINT cli_nombre CHECK (Nombre IS NOT NULL),
    CONSTRAINT cli_apellidos CHECK (Apellidos IS NOT NULL),
    CONSTRAINT cli_edad CHECK (Edad IS NOT NULL),
    CONSTRAINT cli_direccion CHECK (Direccion IS NOT NULL),
    CONSTRAINT cli_telefono CHECK (Telefono IS NOT NULL),

    -- Restricciones de valor
    CONSTRAINT cli_edad_valida CHECK (Edad >= 18),
    CONSTRAINT cli_email_valido CHECK (Email LIKE '%@%.%'),
    CONSTRAINT cli_dni_valido CHECK (REGEXP_LIKE(Dni, '^[0-9]{8}[a-zA-Z]$')),

    ref is id_cliente system generated
);

CREATE TABLE Cuenta of CuentaUdt (
    PRIMARY KEY (Iban),
    CONSTRAINT cue_numero CHECK (Numero IS NOT NULL),
    CONSTRAINT cue_fecha CHECK (Fecha_creacion IS NOT NULL),
    CONSTRAINT cue_saldo CHECK (Saldo_actual IS NOT NULL),

    -- Restricciones de valor
    CONSTRAINT cue_saldo_positivo CHECK (Saldo_actual >= 0),

    ref is id_cuenta system generated
);

CREATE TABLE Operacion of OperacionUdt (
    PRIMARY KEY (Codigo, refCuentaOrigen),
    CONSTRAINT op_fecha CHECK (Fecha IS NOT NULL),
    CONSTRAINT op_cantidad CHECK (Cantidad IS NOT NULL),
    CONSTRAINT op_cuenta_orig CHECK (refCuentaOrigen IS NOT NULL),

    -- Restricciones de valor
    CONSTRAINT op_cantidad_pos CHECK (Cantidad > 0),

    ref is id_operacion system generated
);

-- =====
-- 2. TABLAS HIJAS (Heredan el REF y La PK)

```



```
-- =====

CREATE TABLE CuentaAhorro of AhorroUdt under Cuenta (
    CONSTRAINT cue_aho_interes CHECK (Tipo_interes IS NOT NULL),
    CONSTRAINT cue_aho_interes_pos CHECK (Tipo_interes > 0)
);

CREATE TABLE CuentaCorriente of CorrienteUdt under Cuenta (
    CONSTRAINT cue_cor_oficina CHECK (refOficina IS NOT NULL)
);

CREATE TABLE Transferencia of TransferenciaUdt under Operacion (
    CONSTRAINT trans_cuenta_dest CHECK (refCuentaDestino IS NOT NULL),

    -- Validamos que no se transfiera a sí misma
    CONSTRAINT trans_cuentas_distintas CHECK (refCuentaOrigen <>
refCuentaDestino)
);

CREATE TABLE OperacionEfectivo of EfectivoUdt under Operacion (
    CONSTRAINT efec_oficina CHECK (refOficina IS NOT NULL)
);
```

5.3. Restricciones adicionales del modelo

Aunque muchas reglas del dominio pueden expresarse directamente mediante la definición de tipos y tablas (restricciones CHECK, obligatoriedad de atributos o acciones referenciales sobre referencias REF), existen otras restricciones que no pueden garantizarse únicamente mediante las sentencias CREATE TYPE o CREATE TABLE. Algunas de estas restricciones dependen del estado global del sistema, de la consistencia entre colecciones de referencias o de la ejecución periódica de rutinas funcionales.

Por esta razón, determinadas reglas deben implementarse mediante mecanismos adicionales del SGBD, como triggers, procedimientos almacenados o tareas programadas. A continuación se enumeran las restricciones adicionales consideradas en el diseño.

R1. No se permiten instancias “genéricas” de Cuenta ni de Operación: toda ocurrencia debe pertenecer a uno de sus subtipos definidos. (En el esquema actual los tipos base son INSTANTIABLE, por lo que esta totalidad no queda impuesta solo por el CREATE TYPE.)

R2. Si un cliente contiene una referencia a una cuenta en su colección de cuentas, dicha cuenta debe contener obligatoriamente una referencia a ese cliente en su colección de titulares, garantizando integridad bidireccional Cliente–Cuenta.

R3. En las colecciones de referencias entre clientes y cuentas no se permiten elementos repetidos: un mismo cliente no puede figurar duplicado como titular de una misma cuenta, ni una cuenta puede aparecer duplicada en la colección de cuentas de un cliente.

R4. La fecha de creación de una cuenta y la fecha/hora de una operación deben ser anteriores o iguales al instante actual, evitando altas u operaciones en el futuro.

R5. La fecha/hora de cualquier operación debe ser posterior o igual a la fecha de creación de la cuenta origen asociada a dicha operación, evitando operaciones sobre cuentas inexistentes en el momento del movimiento.

R6. En las transferencias, la fecha/hora de la operación debe ser posterior o igual a la fecha de creación de la cuenta destino (receptora).

R7. El saldo de las cuentas de ahorro debe incrementarse periódicamente en función de su tipo de interés, conforme al procedimiento de actualización definido (proceso periódico).

R8. El saldo actual de una cuenta debe ser consistente con el histórico de operaciones asociadas (ingresos, retiradas, transferencias emitidas/recibidas e intereses en cuentas de ahorro), por tratarse de un atributo derivado del historial.

R9. Si una cuenta corriente referencia una oficina, dicha oficina debe incluir la cuenta en su colección de cuentas corrientes asociadas, garantizando la consistencia bidireccional Oficina–CuentaCorriente.

R10. Para toda operación realizada en efectivo que referencia una oficina, dicha oficina debe incluir el identificador de la operación en su colección de operaciones en efectivo asociadas, garantizando la consistencia bidireccional Oficina–OperaciónEfectivo.

6. Implementación con modelo objeto/relacional

Una vez definido el diseño lógico objeto-relacional, se procede a su implementación en distintos SGBD que ofrecen soporte, total o parcial, para este tipo de estructuras. Para ello se han desarrollado scripts específicos para cada gestor para representar el modelo diseñado en el apartado anterior. En los siguientes subapartados se describe el procedimiento seguido para ejecutar dichos scripts en cada uno de los sistemas utilizados.

6.1. Oracle (bd-bancaria-or-oracle.sql)

La implementación del modelo objeto-relacional en **Oracle** se realiza mediante el script bd-bancaria-or-oracle.sql. Este script incluye la definición de los tipos objeto, las jerarquías de herencia entre tipos y las tablas tipadas correspondientes.

Siguiendo el mismo procedimiento empleado en apartados anteriores, el contenedor de Oracle se inicia mediante docker-compose. Posteriormente, el script se copia al directorio temporal del contenedor y se ejecuta utilizando la herramienta SQL*Plus. Durante la ejecución del script, Oracle muestra distintos mensajes indicando la creación de los tipos, tablas, triggers y procedimientos definidos en el modelo.

```
root@core ~/bd2/sql/oracle# docker cp bd-bancaria-or-oracle.sql oracle-xe:/tmp/  
Successfully copied 10.2kB to oracle-xe:/tmp/
```

```
root@core ~/bd2/sql/oracle# docker exec -it oracle-xe sqlplus system/oracle123
@/tmp/bd-bancaria-or-oracle.sql
```

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mon Mar 9 18:08:27 2026
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Last Successful login time: Mon Mar 09 2026 18:07:05 +00:00
```

```
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

```
PL/SQL procedure successfully completed.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.
```

```
Type created.  
Table created.  
Table created.  
Table created.  
Table created.  
Trigger created.  
Procedure created.  
PL/SQL procedure successfully completed.  
SQL>
```

6.2. PostgreSQL (bd-bancaria-or-postgresql.sql)

En el caso de **PostgreSQL**, la implementación se realiza mediante el script `bd-bancaria-or-postgresql.sql`. Este script adapta el modelo objeto-relacional al soporte que ofrece PostgreSQL, utilizando mecanismos como tablas heredadas, funciones y triggers para reproducir la lógica definida en el diseño.

El procedimiento de ejecución es similar al utilizado en otros apartados: el script se copia al contenedor Docker que ejecuta PostgreSQL y posteriormente se ejecuta mediante la herramienta `psql`. Durante la ejecución se muestran mensajes informativos indicando la eliminación de tablas previas (cuando existen) y la creación de las nuevas estructuras de la base de datos.

```
root@core ~/bd2/sql/postgresql# docker cp bd-bancaria-or-postgresql.sql  
postgres-practica1:/tmp/  
Successfully copied 6.66kB to postgres-practica1:/tmp/  
  
root@core ~/bd2/sql/postgresql# docker exec -it -e PGPASSWORD=alumno123  
postgres-practica1 psql -U alumno -d postgres -f  
/tmp/bd-bancaria-or-postgresql.sql  
DROP TABLE  
DROP TABLE  
DROP TABLE  
DROP TABLE  
DROP TABLE  
DROP TABLE  
DROP TABLE  
DROP TABLE  
DROP TABLE  
CREATE TABLE  
CREATE TABLE
```

```

CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE PROCEDURE

```

6.3. DB2 (bd-bancaria-or-db2.sql)

La implementación del modelo objeto-relacional en **IBM Db2** se realiza mediante el script `bd-bancaria-or-db2.sql`. En este caso se emplean tipos definidos por el usuario y tablas tipadas para representar las entidades y jerarquías del modelo, siguiendo las capacidades objeto-relacionales que ofrece este sistema gestor.

El script se copia al contenedor Docker que ejecuta Db2 y posteriormente se ejecuta mediante la herramienta de línea de comandos `db2`. Durante su ejecución se eliminan previamente los tipos y tablas existentes, permitiendo que el script pueda ejecutarse repetidamente sin conflictos, y se crean posteriormente los tipos, las jerarquías de tablas y las tablas tipadas correspondientes. La salida generada por Db2 muestra los mensajes asociados a la creación de los distintos tipos y tablas del sistema.

```

root@core ~/bd2/sql/db2# docker cp bd-bancaria-or-db2.sql contenedor_db2:/tmp/
Successfully copied 5.63kB to contenedor_db2:/tmp/

root@core ~/bd2/sql/db2# docker exec -it contenedor_db2 su - db2inst1 -c "db2
-t@ -vf /tmp/bd-bancaria-or-db2.sql"
CONNECT TO DB2INST1

Database Connection Information

Database server          = DB2/LINUX8664 11.5.8.0
SQL authorization ID    = DB2INST1
Local database alias    = DB2INST1

BEGIN
DECLARE CONTINUE HANDLER FOR SQLSTATE '42704' BEGIN END;
DECLARE CONTINUE HANDLER FOR SQLSTATE '42809' BEGIN END; -- Maneja el error si
la jerarquía no existe aún

-- Borrado de jerarquías de tablas tipadas (Sintaxis corregida)
EXECUTE IMMEDIATE 'DROP TABLE HIERARCHY Operacion';
EXECUTE IMMEDIATE 'DROP TABLE HIERARCHY Cuenta';

```

```
EXECUTE IMMEDIATE 'DROP TABLE Cliente';
EXECUTE IMMEDIATE 'DROP TABLE Oficina';

-- Borrado de tipos
EXECUTE IMMEDIATE 'DROP TYPE EfectivoUdt';
EXECUTE IMMEDIATE 'DROP TYPE TransferenciaUdt';
EXECUTE IMMEDIATE 'DROP TYPE OperacionUdt';
EXECUTE IMMEDIATE 'DROP TYPE CorrienteUdt';
EXECUTE IMMEDIATE 'DROP TYPE AhorroUdt';
EXECUTE IMMEDIATE 'DROP TYPE CuentaUdt';
EXECUTE IMMEDIATE 'DROP TYPE ClienteUdt';
EXECUTE IMMEDIATE 'DROP TYPE OficinaUdt';
END

DB20000I The SQL command completed successfully.

CREATE TYPE OficinaUdt AS ( Codigo DECIMAL(5,0), Telefono DECIMAL(9,0),
Direccion VARCHAR(100) ) MODE DB2SQL INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE ClienteUdt AS ( Dni VARCHAR(9), Nombre VARCHAR(50), Email
VARCHAR(50), Apellidos VARCHAR(50), Edad DECIMAL(3,0), Direccion VARCHAR(100),
Telefono DECIMAL(9,0) ) MODE DB2SQL INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE CuentaUdt AS ( Iban VARCHAR(24), Numero DECIMAL(20,0),
Fecha_creacion DATE, Saldo_actual DECIMAL(10,2) ) MODE DB2SQL INSTANTIABLE NOT
FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE AhorroUdt UNDER CuentaUdt AS ( Tipo_interes DECIMAL(10,2) ) MODE
DB2SQL INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE CorrienteUdt UNDER CuentaUdt AS ( refOficina DECIMAL(5,0) ) MODE
DB2SQL INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE OperacionUdt AS ( Codigo DECIMAL(8,0), Concepto VARCHAR(250), Fecha
TIMESTAMP, Cantidad DECIMAL(10,2), refCuentaOrigen VARCHAR(24) ) MODE DB2SQL
INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE TransferenciaUdt UNDER OperacionUdt AS ( refCuentaDestino
VARCHAR(24) ) MODE DB2SQL INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.

CREATE TYPE EfectivoUdt UNDER OperacionUdt AS ( refOficina DECIMAL(5,0) ) MODE
DB2SQL INSTANTIABLE NOT FINAL
DB20000I The SQL command completed successfully.
```

```
CREATE TABLE Oficina OF OficinaUdt ( REF IS oid_oficina USER GENERATED, Codigo
WITH OPTIONS NOT NULL PRIMARY KEY, Telefono WITH OPTIONS NOT NULL, Direccion
WITH OPTIONS NOT NULL )
DB20000I The SQL command completed successfully.

CREATE TABLE Cliente OF ClienteUdt ( REF IS oid_cliente USER GENERATED, Dni WITH
OPTIONS NOT NULL PRIMARY KEY, Nombre WITH OPTIONS NOT NULL, Apellidos WITH
OPTIONS NOT NULL, Edad WITH OPTIONS NOT NULL CHECK (Edad >= 18), Direccion WITH
OPTIONS NOT NULL, Telefono WITH OPTIONS NOT NULL )
DB20000I The SQL command completed successfully.

CREATE TABLE Cuenta OF CuentaUdt ( REF IS oid_cuenta USER GENERATED, Iban WITH
OPTIONS NOT NULL PRIMARY KEY, Numero WITH OPTIONS NOT NULL, Fecha_creacion WITH
OPTIONS NOT NULL, Saldo_actual WITH OPTIONS NOT NULL CHECK (Saldo_actual >= 0) )
DB20000I The SQL command completed successfully.

CREATE TABLE CuentaAhorro OF AhorroUdt UNDER Cuenta INHERIT SELECT PRIVILEGES (
Tipo_interes WITH OPTIONS NOT NULL CHECK (Tipo_interes > 0) )
DB20000I The SQL command completed successfully.

CREATE TABLE CuentaCorriente OF CorrienteUdt UNDER Cuenta INHERIT SELECT
PRIVILEGES ( refOficina WITH OPTIONS NOT NULL )
DB20000I The SQL command completed successfully.

CREATE TABLE Operacion OF OperacionUdt ( REF IS oid_operacion USER GENERATED,
Codigo WITH OPTIONS NOT NULL PRIMARY KEY, Fecha WITH OPTIONS NOT NULL, Cantidad
WITH OPTIONS NOT NULL CHECK (Cantidad > 0), refCuentaOrigen WITH OPTIONS NOT
NULL )
DB20000I The SQL command completed successfully.

CREATE TABLE Transferencia OF TransferenciaUdt UNDER Operacion INHERIT SELECT
PRIVILEGES ( refCuentaDestino WITH OPTIONS NOT NULL )
DB20000I The SQL command completed successfully.

CREATE TABLE OperacionEfectivo OF EfectivoUdt UNDER Operacion INHERIT SELECT
PRIVILEGES ( refOficina WITH OPTIONS NOT NULL )
DB20000I The SQL command completed successfully.

CONNECT RESET
DB20000I The SQL command completed successfully.
```

7. Generación de datos y pruebas

Una vez implementado el modelo objeto-relacional en los distintos SGBD, se procede a realizar un conjunto de pruebas destinadas a verificar el correcto funcionamiento del esquema definido y de las reglas de negocio asociadas. Para ello se han desarrollado scripts específicos de inserción de datos y pruebas que permiten comprobar tanto el comportamiento del sistema ante datos válidos como la detección de situaciones que violan las restricciones definidas en el modelo.

Las pruebas realizadas incluyen la inserción de datos iniciales, la verificación de restricciones de integridad, la ejecución de operaciones bancarias típicas (ingresos, retiradas y transferencias) y la comprobación de los mecanismos automáticos de actualización del saldo de las cuentas. En los siguientes subapartados se describe la ejecución de estos scripts en cada uno de los sistemas gestores utilizados.

7.1. Oracle (bd-bancaria-or-pruebas-oracle.sql)

En el caso de **Oracle**, las pruebas se realizan mediante el script `bd-bancaria-or-pruebas-oracle.sql`. Este script se copia previamente al contenedor Docker que ejecuta Oracle y posteriormente se ejecuta utilizando la herramienta SQL*Plus, de forma análoga a los scripts de creación de la base de datos.

El script incluye distintos bloques de pruebas. En primer lugar se insertan datos base válidos correspondientes a clientes, oficinas y cuentas, lo que permite inicializar el sistema y comprobar que las relaciones entre entidades funcionan correctamente. A continuación se realizan varias pruebas destinadas a verificar que las restricciones de integridad se aplican correctamente, provocando errores cuando se intenta insertar información inválida, como DNIs con formato incorrecto o clientes menores de edad.

Posteriormente se ejecutan operaciones bancarias válidas para comprobar el funcionamiento de los triggers encargados de actualizar el saldo de las cuentas tras cada operación. Finalmente, se incluyen pruebas relacionadas con situaciones de fondos insuficientes y con la ejecución del procedimiento encargado de calcular los intereses de las cuentas de ahorro.

```
root@core ~/bd2/sql/oracle# docker cp bd-bancaria-or-pruebas-oracle.sql
oracle-xe:/tmp/
Successfully copied 8.19kB to oracle-xe:/tmp/

root@core ~/bd2/sql/oracle# docker exec -it oracle-xe sqlplus system/oracle123
@/tmp/bd-bancaria-or-pruebas-oracle.sql

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Mar 9 18:12:35 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Mar 09 2026 18:08:28 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

Session altered.

=====
0. LIMPIEZA DE DATOS PREVIOS
=====

0 rows deleted.

0 rows deleted.
```



```
0 rows deleted.

0 rows deleted.

Commit complete.

Datos anteriores eliminados correctamente.
=====
1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE)
=====

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row updated.

1 row updated.

Commit complete.

Datos base insertados correctamente.
=====
2. PRUEBAS DE RESTRICCIONES DE INTEGRIDAD (ERRORES ORA-)
=====
> Prueba 2.1: DNI con formato erroneo (Falla por CHECK cli_dni_chk)
INSERT INTO Cliente (Dni, Nombre, Email, Apellidos, Edad, Direccion, Telefono,
refCuenta)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CLI_DNI_CHK) violated

> Prueba 2.2: Cliente menor de edad (Falla por CHECK cli_edad_val)
INSERT INTO Cliente (Dni, Nombre, Email, Apellidos, Edad, Direccion, Telefono,
refCuenta)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CLI_EDAD_VAL) violated

> Prueba 2.3: Operacion con fecha anterior a creacion de la cuenta (Falla por
Trigger TRG_VAL_FECHAS_OP)
INSERT INTO Operacion VALUES (
*
ERROR at line 1:
ORA-20002: Fecha operaci??n anterior a apertura cuenta origen.
ORA-06512: at "SYSTEM.TRG_OPERACIONES_COMPUESTO", line 21
ORA-04088: error during execution of trigger 'SYSTEM.TRG_OPERACIONES_COMPUESTO'
```

```

=====
3. PRUEBAS DE OPERACIONES Y TRIGGERS DE SALDO (DEBEN FUNCIONAR)
=====
> 3.1: Hacemos un Ingreso de 1000 euros en la cuenta de Ana (Operacion en
Efectivo)

1 row updated.

Ingreso manual inicializado para pruebas de extraccion.
> 3.2: Hacemos una Retirada de 200 euros en la cuenta de Ana (Operacion en
Efectivo)

1 row created.

> 3.3: Ana hace una transferencia de 300 euros a Luis

1 row created.

> ESTADO DE SALDOS ESPERADO:
> Ana empezo con 1000 - 200 - 300 = 500 euros.
> Luis empezo con 0 + 300 transferidos = 300 euros.

IBAN                                SALDO_ACTUAL
-----
ES9100001111222233334444          500
ES9100005555666677778888          300

=====
4. PRUEBA DE DISPARADOR DE FONDOS INSUFICIENTES (CHECK SALDO)
=====
> Ana intenta sacar 600 euros pero solo tiene 500.
> El trigger intentara restar el saldo y el CHECK (Saldo_actual >= 0) de la
tabla Cuenta lo impedira.
INSERT INTO Operacion VALUES (
    *
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CUE_SALDO_CHK) violated
ORA-06512: at "SYSTEM.TRG_OPERACIONES_COMPUESTO", line 25
ORA-04088: error during execution of trigger 'SYSTEM.TRG_OPERACIONES_COMPUESTO'

> Comprobamos que el saldo de Ana sigue intacto en 500 euros

IBAN                                SALDO_ACTUAL
-----
ES9100001111222233334444          500

=====
5. PRUEBA DEL PROCEDIMIENTO DE INTERESES NOCTURNOS
=====
> Ejecutamos el pago de intereses manual (Liquidacion de la cuenta de ahorro de
Luis al 2.5%)

PL/SQL procedure successfully completed.

> Comprobamos que el saldo de Luis ha subido (Tenia 300, y se le ha sumado su
interes)

```

IBAN	SALDO_ACTUAL
ES9100005555666677778888	307.5

Commit complete.

PRUEBAS FINALIZADAS.
SQL>

7.2. PostgreSQL (bd-bancaria-or-pruebas-postgresql.sql)

Para **PostgreSQL**, las pruebas se realizan mediante el script `bd-bancaria-or-pruebas-postgresql.sql`, que se copia previamente al contenedor Docker correspondiente y se ejecuta utilizando la herramienta `psql`. Este script permite comprobar el correcto funcionamiento de las tablas, funciones y triggers definidos en la implementación objeto-relacional.

Al igual que en el caso anterior, el script se organiza en varios bloques de pruebas. En primer lugar se insertan datos válidos que permiten inicializar el sistema. Posteriormente se ejecutan pruebas de restricciones destinadas a verificar que el sistema detecta correctamente situaciones inválidas, como correos electrónicos con formato incorrecto o clientes menores de edad.

También se realizan operaciones bancarias sobre las cuentas para comprobar el funcionamiento de los triggers encargados de actualizar los saldos, así como pruebas destinadas a verificar que el sistema impide operaciones que provocarían un saldo negativo. Finalmente, se ejecuta el procedimiento encargado de calcular los intereses de las cuentas de ahorro y se comprueba que el saldo resultante se actualiza correctamente.

```
root@core ~/bd2/sql/postgresql# docker cp bd-bancaria-or-pruebas-postgresql.sql
postgres-practica1:/tmp/
Successfully copied 7.17kB to postgres-practica1:/tmp/

root@core ~/bd2/sql/postgresql# docker exec -it -e PGPASSWORD=alumno123
postgres-practica1 psql -U alumno -d postgres -f
/tmp/bd-bancaria-or-pruebas-postgresql.sql
=====
0. LIMPIEZA DE DATOS PREVIOS
=====
TRUNCATE TABLE
TRUNCATE TABLE
TRUNCATE TABLE
TRUNCATE TABLE
Datos anteriores eliminados correctamente.
=====
1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE)
=====
INSERT 0 1
INSERT 0 1
```

```

INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
UPDATE 1
UPDATE 1
Datos base insertados correctamente.
=====
2. PRUEBAS DE RESTRICCIONES DE INTEGRIDAD (ERRORES ESPERADOS)
=====
> Prueba 2.1: Email con formato erróneo (Falla por CHECK de la tabla)
psql:/tmp/bd-bancaria-or-pruebas-postgresql.sql:52: ERROR:  new row for relation
"cliente" violates check constraint "cliente_email_check"
DETAIL:  Failing row contains (123A45678, Falso, falsoemail.com, Dni, 30, Dir,
600000000, {}).
> Prueba 2.2: Cliente menor de edad (Falla por CHECK de la tabla)
psql:/tmp/bd-bancaria-or-pruebas-postgresql.sql:56: ERROR:  new row for relation
"cliente" violates check constraint "cliente_edad_check"
DETAIL:  Failing row contains (11111111C, Joven, joven@email.com, Menor, 16,
Dir, 600000000, {}).
> Prueba 2.3: Operación con fecha anterior a creación de cuenta (Falla por
Trigger PL/pgSQL)
psql:/tmp/bd-bancaria-or-pruebas-postgresql.sql:60: ERROR:  Fecha operación
anterior a apertura cuenta origen.
CONTEXT:  PL/pgSQL function func_logica_efectivo() line 8 at RAISE
=====
3. PRUEBAS DE OPERACIONES Y TRIGGERS DE SALDO
=====
> 3.1: Inicializamos el saldo de Ana a 1000 euros para pruebas
UPDATE 1
> 3.2: Hacemos una Retirada de 200 euros en la cuenta de Ana (OperacionEfectivo)
INSERT 0 1
> 3.3: Ana hace una transferencia de 300 euros a Luis (Transferencia)
INSERT 0 1
> ESTADO DE SALDOS ESPERADO:
> Ana: 1000 - 200 - 300 = 500 euros.
> Luis: 0 + 300 transferidos = 300 euros.
      iban          | saldo_actual
-----+-----
ES91000055556667778888 |      300.00
ES9100001111222233334444 |      500.00
(2 rows)

=====
4. PRUEBA DE DISPARADOR DE FONDOS INSUFICIENTES (CHECK SALDO)
=====
> Ana intenta sacar 600 euros pero solo tiene 500. El CHECK (Saldo_actual >= 0)
lo impedirá.
psql:/tmp/bd-bancaria-or-pruebas-postgresql.sql:89: ERROR:  new row for relation
"cuentacorrente" violates check constraint "cuenta_saldo_actual_check"

```

```

DETAIL: Failing row contains (ES9100001111222233334444, 1111222233334444,
2026-03-09 18:13:45.91136, -100.00, {12345678A}).
CONTEXT: SQL statement "UPDATE Cuenta SET Saldo_actual = Saldo_actual -
NEW.Cantidad WHERE Iban = NEW.refCuentaOrigen"
PL/pgSQL function func_logica_efectivo() line 12 at SQL statement
> Comprobamos que el saldo de Ana sigue intacto en 500 euros
      iban          | saldo_actual
-----+-----
 ES9100001111222233334444 |      500.00
(1 row)

=====
5. PRUEBA DEL PROCEDIMIENTO DE INTERESES NOCTURNOS
=====
> Ejecutamos el pago de intereses (Liquidación de la cuenta de ahorro de Luis al
2.5%)
CALL
> Comprobamos que el saldo de Luis ha subido (Tenía 300, se le suma el 2.5% =
307.50)
      iban          | saldo_actual
-----+-----
 ES9100005555666677778888 |      307.50
(1 row)

psql:/tmp/bd-bancaria-or-pruebas-postgresql.sql:104: error: unterminated quoted
string

```

7.3. DB2 (bd-bancaria-or-pruebas-db2.sql)

En el caso de **IBM Db2**, las pruebas se realizan mediante el script `bd-bancaria-or-pruebas-db2.sql`. Este script se copia al contenedor Docker que ejecuta Db2 y se ejecuta utilizando la herramienta de línea de comandos `db2`.

Antes de insertar los datos de prueba, el script elimina los registros existentes en las distintas tablas del esquema, lo que permite ejecutar las pruebas repetidamente sin generar conflictos con datos previamente insertados. Posteriormente se insertan datos base correspondientes a oficinas, clientes y cuentas.

A continuación se realizan distintas pruebas de integridad destinadas a comprobar que el sistema detecta correctamente situaciones inválidas, como clientes menores de edad o cuentas con saldo negativo. Finalmente, se ejecutan operaciones bancarias válidas para verificar el correcto funcionamiento del modelo, comprobando que los saldos de las cuentas se actualizan según las operaciones realizadas.

```

root@core ~/bd2/sql/db2# docker cp bd-bancaria-or-pruebas-db2.sql
contenedor_db2:/tmp/
Successfully copied 5.12kB to contenedor_db2:/tmp/

root@core ~/bd2/sql/db2# docker exec -it contenedor_db2 su - db2inst1 -c "db2

```

```
-td@ -vf /tmp/bd-bancaria-or-pruebas-db2.sql"
```

```
CONNECT TO DB2INST1
```

Database Connection Information

```
Database server      = DB2/LINUX8664 11.5.8.0
SQL authorization ID  = DB2INST1
Local database alias  = DB2INST1
```

```
DELETE FROM Operacion
```

```
SQL0100W No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
```

```
DELETE FROM Cuenta
```

```
SQL0100W No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
```

```
DELETE FROM Cliente
```

```
SQL0100W No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
```

```
DELETE FROM Oficina
```

```
SQL0100W No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
```

```
COMMIT
```

```
DB20000I The SQL command completed successfully.
```

```
INSERT INTO Oficina (oid_oficina, Codigo, Telefono, Direccion) VALUES
(OficinaUdt('1'), 1, 976111222, 'Paseo Independencia, Zaragoza')
```

```
DB20000I The SQL command completed successfully.
```

```
INSERT INTO Oficina (oid_oficina, Codigo, Telefono, Direccion) VALUES
(OficinaUdt('2'), 2, 911222333, 'Gran Vía, Madrid')
```

```
DB20000I The SQL command completed successfully.
```

```
INSERT INTO Cliente (oid_cliente, Dni, Nombre, Email, Apellidos, Edad,
Direccion, Telefono) VALUES (ClienteUdt('1'), '12345678A', 'Ana',
'ana@email.com', 'García', 25, 'Calle Mayor 1', 600111222)
```

```
DB20000I The SQL command completed successfully.
```

```
INSERT INTO Cliente (oid_cliente, Dni, Nombre, Email, Apellidos, Edad,
Direccion, Telefono) VALUES (ClienteUdt('2'), '87654321B', 'Luis',
'luis@email.com', 'Pérez', 40, 'Calle Menor 2', 600333444)
```

```
DB20000I The SQL command completed successfully.
```

```
INSERT INTO CuentaCorriente (oid_cuenta, Iban, Numero, Fecha_creacion,
Saldo_actual, refOficina) VALUES (CorrienteUdt('1'), 'ES9100001111222233334444',
1111222233334444, CURRENT DATE, 0, 1)
```

DB20000I The SQL command completed successfully.

```
INSERT INTO CuentaAhorro (oid_cuenta, Iban, Numero, Fecha_creacion,
Saldo_actual, Tipo_interes) VALUES (AhorroUdt('2'), 'ES9100005555666677778888',
5555666677778888, CURRENT DATE, 0, 2.5)
```

DB20000I The SQL command completed successfully.

COMMIT

DB20000I The SQL command completed successfully.

```
INSERT INTO Cliente (oid_cliente, Dni, Nombre, Apellidos, Email, Edad,
Direccion, Telefono) VALUES (ClienteUdt('3'),
'11111111C', 'Joven', 'Menor', 'joven@email.com', 16, 'Dir', 600000000)
```

DB21034E The command was processed as an SQL statement because it was not a valid Command Line Processor command. During SQL processing it returned:

SQL0545N The requested operation is not allowed because a row does not satisfy the check constraint "DB2INST1.CLIENTE.SQL260309181155740".

SQLSTATE=23513

```
INSERT INTO Cuenta (oid_cuenta, Iban, Numero, Fecha_creacion, Saldo_actual)
VALUES (CuentaUdt('3'), 'ES000000000000000000000000', 1111, CURRENT DATE, -100)
```

DB21034E The command was processed as an SQL statement because it was not a valid Command Line Processor command. During SQL processing it returned:

SQL0545N The requested operation is not allowed because a row does not satisfy the check constraint "DB2INST1.CUENTA.SQL260309181156090".

SQLSTATE=23513

```
UPDATE Cuenta SET Saldo_actual = 1000 WHERE Iban = 'ES9100001111222233334444'
```

DB20000I The SQL command completed successfully.

```
INSERT INTO OperacionEfectivo (oid_operacion,Codigo, Concepto, Fecha, Cantidad,
refCuentaOrigen, refOficina) VALUES (EfectivoUdt('1'), 1, 'Cajero', CURRENT
TIMESTAMP, 200, 'ES9100001111222233334444', 1)
```

DB20000I The SQL command completed successfully.

```
INSERT INTO Transferencia (oid_operacion, Codigo, Concepto, Fecha, Cantidad,
refCuentaOrigen, refCuentaDestino) VALUES (TransferenciaUdt('2'), 2, 'Regalo',
CURRENT TIMESTAMP, 300, 'ES9100001111222233334444', 'ES9100005555666677778888')
```

DB20000I The SQL command completed successfully.

```
UPDATE Cuenta SET Saldo_actual = Saldo_actual - 200 - 300 WHERE Iban =
'ES9100001111222233334444'
```

DB20000I The SQL command completed successfully.

```
UPDATE Cuenta SET Saldo_actual = Saldo_actual + 300 WHERE Iban =
'ES9100005555666677778888'
```

DB20000I The SQL command completed successfully.

```
UPDATE Cuenta SET Saldo_actual = Saldo_actual - 600 WHERE Iban =
'ES9100001111222233334444'
```

```
DB21034E The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0545N The requested operation is not allowed because a row does not
satisfy the check constraint "DB2INST1.CUENTA.SQL260309181156090".
SQLSTATE=23513
```

```
SELECT Iban, Saldo_actual FROM Cuenta
```

IBAN	SALDO_ACTUAL
ES9100001111222233334444	500.00
ES9100005555666677778888	300.00

```
2 record(s) selected.
```

```
CONNECT RESET
```

```
DB20000I The SQL command completed successfully.
```

8. Implementación con db4o (Java)

En esta última parte de la práctica se realiza una implementación del sistema utilizando **db4o (database for objects)**, una base de datos orientada a objetos que permite almacenar directamente objetos Java sin necesidad de transformarlos previamente a estructuras relacionales.

Para llevar a cabo esta implementación se utilizó el **ejercicio de ejemplo proporcionado en Moodle**, en el cual se incluye la biblioteca necesaria para trabajar con db4o. En particular, dentro de la carpeta LIB se encuentra el fichero db4o-8.0.249.16098-all-java5.jar, que contiene las clases necesarias para utilizar la base de datos desde aplicaciones Java.

A diferencia de las implementaciones anteriores, basadas en SQL y en esquemas de tablas, en este caso el modelo se representa directamente mediante **clases Java**, donde **cada clase corresponde a una entidad del dominio** del problema (por ejemplo, Cliente, Cuenta, Oficina u Operacion). Estas clases incluyen tanto los atributos de los objetos como los métodos necesarios para validar determinadas restricciones del modelo.

Todos los **ficheros fuente (.java)** se almacenaron en una misma carpeta del proyecto, denominada **DB4o**, desde la cual se compila y ejecuta la aplicación.

8.1. Compilación y ejecución

Una vez definidos los distintos ficheros Java, el código **se compila desde la línea de comandos** utilizando el **compilador de Java (javac)**, indicando en el classpath la biblioteca de db4o. Posteriormente **se ejecuta** la aplicación **mediante la clase principal Main**, que contiene las operaciones de prueba sobre la base de datos.

Los comandos utilizados fueron los siguientes. El **primer comando** compila todos los archivos .java presentes en el directorio, mientras que el **segundo comando** ejecuta el programa principal incluyendo la biblioteca de db4o en el entorno de ejecución.


```
PS D:\CUARTO\BASES2\PRACTICAS\PRACTICA2\DB4o> javac -cp
";db4o-8.0.249.16098-all-java5.jar" *.java

PS D:\CUARTO\BASES2\PRACTICAS\PRACTICA2\DB4o> java --add-opens
java.base/java.lang=ALL-UNNAMED -cp ";db4o-8.0.249.16098-all-java5.jar" Main

--- 0. RESETEANDO BASE DE DATOS ---

--- 1. CREANDO DATOS ---
OK: Datos base insertados.

--- 2. BUSCANDO CLIENTE POR DNI (12345678A) ---
¡Encontrada!: Ana García

--- 3. ACTUALIZANDO SALDO DE CUENTA CORRIENTE ---
Saldo anterior: 1000.0
Saldo nuevo: 1500.0

--- 4. BORRANDO A LUIS PÉREZ ---
Luis eliminado correctamente.

--- 5. ESTADO FINAL ---
Clientes en BD: 1
- Ana García

--- 6. PROBANDO INTEGRIDAD: Borrar Oficina de un Efectivo ---
Paso A: Oficina y Efectivo guardados.
Paso B: Oficina borrada de la BD.
RESULTADO: El objeto aún tiene la referencia (Sigue en memoria RAM).

--- 7. PRUEBAS DE RESTRICCIONES (LOGS DE ERROR) ---
Probando DNI mal formado... CAPTURADO: El DNI no es valido, debe estar formado
por 8 numeros y una letra
Probando Edad < 18... CAPTURADO: La edad minima del cliente debe ser 18 y debe
tener como maximo 3 digitos
Probando Saldo negativo... CAPTURADO: El saldo de la cuenta no puede ser
negativo
Probando Transferencia a la misma cuenta... CAPTURADO: No se puede transferir
dinero a la misma cuenta
Probando Operación en fecha anterior a la cuenta... CAPTURADO: La fecha en la
operacion no puede ser anterior a la creacion de la cuenta de origen
Probando Oficina con dirección vacía... CAPTURADO: La direccion de la oficina no
puede ser nula

Proceso finalizado.
```

Al ejecutar el programa, la aplicación **comienza reiniciando la base de datos y creando un conjunto de datos iniciales** que permiten realizar las distintas pruebas del sistema. En primer lugar **se insertan objetos correspondientes a clientes, cuentas y oficinas**, comprobándose que los datos se almacenan correctamente en la base de datos. A

continuación se realizan distintas operaciones típicas, como la búsqueda de un cliente por su DNI, la actualización del saldo de una cuenta y la eliminación de un cliente existente.

Estas operaciones permiten **verificar el funcionamiento básico** del sistema orientado a objetos y comprobar que los objetos almacenados en la base de datos **pueden recuperarse, modificarse y eliminarse correctamente**.

Además de las operaciones básicas, el programa incluye un conjunto de pruebas destinadas a **verificar el cumplimiento de determinadas restricciones de integridad**. En este caso, dichas restricciones se implementan mediante validaciones dentro del propio código Java, ya que db4o no dispone de mecanismos declarativos de integridad equivalentes a los de los sistemas relacionales.

Durante la ejecución del programa se realizan **intentos de inserción de datos inválidos**, como DNIs con formato incorrecto, clientes menores de edad, cuentas con saldo negativo o transferencias realizadas a la misma cuenta de origen. En todos estos casos, las excepciones generadas por el programa son capturadas y mostradas por pantalla mediante mensajes informativos.

Asimismo, se realiza una **prueba relacionada con la eliminación de una oficina asociada a una operación en efectivo**, observándose que la referencia al objeto eliminado permanece en memoria mientras el objeto continúa cargado en la aplicación. Este comportamiento muestra una de las diferencias entre las bases de datos orientadas a objetos y los sistemas relacionales, donde las restricciones de integridad referencial se gestionan directamente en el propio gestor de bases de datos.

En conclusión, el conjunto de pruebas realizado permite confirmar que la **implementación en db4o reproduce fielmente el comportamiento definido** para el sistema bancario. A diferencia de los modelos previos, donde las validaciones se delegaban en el propio motor de la base de datos, en este escenario la **responsabilidad recae sobre la lógica de las clases Java**. El correcto funcionamiento de los tests demuestra que es posible garantizar la coherencia de la información mediante una programación rigurosa de los métodos de control, compensando así que el gestor de objetos no dispone de herramientas de validación automática como los sistemas basados en SQL.

9. Comparación de SGBD

A lo largo de la práctica se han utilizado distintos SGBD con el objetivo de implementar un mismo dominio de aplicación utilizando diferentes modelos de datos. En particular, se han empleado **Oracle, PostgreSQL e IBM Db2** para las implementaciones relacionales y objeto-relacionales, así como **db4o** para la implementación orientada a objetos. Esto ha permitido observar algunas diferencias en la forma en que cada gestor soporta estos modelos y en cómo se implementan determinadas funcionalidades.

En cuanto al **modelo relacional**, los tres gestores utilizados ofrecen un soporte completo y bastante similar. En todos ellos es posible definir tablas, claves primarias y ajenas, restricciones de integridad, así como triggers y procedimientos almacenados. Sin embargo, existen **diferencias en el lenguaje utilizado para implementar la lógica**. Por ejemplo, Oracle utiliza **PL/SQL**, PostgreSQL emplea **PL/pgSQL**, y Db2 utiliza **su propia**

variante de SQL. Aunque la funcionalidad final es la misma, la sintaxis y algunos detalles de implementación cambian entre sistemas.

Las diferencias se hacen más evidentes cuando se trabaja con el **modelo objeto-relacional**. En este caso, **Oracle y Db2** ofrecen un soporte más cercano al estándar SQL:1999, permitiendo definir tipos estructurados, jerarquías de tipos y tablas tipadas. Esto facilita representar directamente las relaciones de herencia definidas en el modelo conceptual.

Por su parte, **PostgreSQL** no implementa completamente todas las características del modelo objeto-relacional del estándar. En su lugar, proporciona mecanismos alternativos como la herencia entre tablas, los tipos compuestos y el uso de funciones y triggers para implementar parte de la lógica del modelo. Como consecuencia, la implementación en PostgreSQL requiere en algunos casos adaptar el diseño o añadir lógica adicional.

También se observan diferencias en la forma de **expresar la herencia y los tipos definidos por el usuario (UDT)**. En **Oracle y Db2** la herencia entre tipos se define mediante la cláusula UNDER, lo que permite construir jerarquías de tipos y tablas tipadas de forma explícita. En cambio, **PostgreSQL** utiliza la cláusula INHERITS entre tablas, lo que implica un enfoque algo diferente para representar estas jerarquías.

Finalmente, la **implementación realizada con db4o** muestra un enfoque completamente distinto al de los sistemas relacionales. En este caso los datos no se almacenan en tablas, sino directamente como **objetos Java**, manteniendo referencias entre ellos. Esto simplifica la integración con el código de la aplicación, pero implica que muchas restricciones de integridad deben implementarse manualmente en el propio programa mediante validaciones y excepciones.

En conjunto, la práctica ha permitido comprobar que, aunque el mismo modelo conceptual puede implementarse en distintos gestores, **cada sistema ofrece herramientas y características diferentes**. Por ello, en muchos casos es necesario adaptar la implementación a las capacidades concretas de cada SGBD.

10. Dificultades y lecciones aprendidas

10.1. Diferencias entre SGBD en el soporte objeto-relacional

Una de las dificultades encontradas durante la práctica fue comprobar que, aunque varios sistemas gestores de bases de datos ofrecen soporte para el modelo objeto-relacional, la forma en que este soporte se implementa varía entre ellos. En el caso de Oracle, el soporte objeto-relacional se basa en el uso de tipos objeto (OBJECT TYPES), referencias (REF), colecciones y tablas tipadas. Estos mecanismos permiten definir estructuras complejas y establecer relaciones entre objetos directamente dentro del esquema de la base de datos.

IBM DB2 también ofrece soporte para el modelo objeto-relacional mediante tipos definidos por el usuario (UDT) y tablas tipadas que pueden organizarse en jerarquías utilizando la cláusula UNDER. Este mecanismo permite representar relaciones de herencia entre tipos y tablas, aunque la sintaxis y la forma de definir los objetos difieren de las utilizadas en Oracle.

En PostgreSQL la implementación es diferente. Aunque el sistema incluye algunas características relacionadas con el paradigma objeto-relacional, como la herencia de tablas (INHERITS), estas no reproducen exactamente el mismo comportamiento que en los otros gestores. En este caso fue necesario adaptar la implementación utilizando mecanismos propios del sistema, como arrays y funciones en PL/pgSQL.

Esta diferencia entre sistemas gestores implica que un mismo diseño conceptual no puede trasladarse de forma idéntica entre distintos SGBD. Aunque el modelo conceptual de la base de datos es común, la implementación debe adaptarse a las características de cada gestor.

10.2. Gestión de dependencias entre objetos

Durante el desarrollo de los scripts de creación del esquema surgió la dificultad de gestionar correctamente las dependencias entre los distintos objetos de la base de datos. En particular, la presencia de tipos definidos por el usuario, jerarquías de tipos y tablas tipadas genera dependencias entre objetos que impiden eliminarlos directamente cuando existen otros objetos que dependen de ellos. Esta situación resulta especialmente relevante en los modelos objeto-relacionales implementados en Oracle y DB2.

Para poder ejecutar los scripts varias veces sin errores fue necesario definir una fase inicial de limpieza en la que se eliminan los objetos existentes siguiendo un orden adecuado, teniendo en cuenta dichas dependencias. En algunos casos también fue necesario utilizar bloques BEGIN ... EXCEPTION o manejadores de errores para evitar que la eliminación de objetos inexistentes provocase la interrupción del script.

Esta situación muestra la importancia de considerar el orden de creación y eliminación de los objetos dentro de un sistema gestor de bases de datos, especialmente cuando existen dependencias entre ellos.

10.3. Ejecución de scripts dentro de contenedores Docker

Otra dificultad encontrada estuvo relacionada con la ejecución de los scripts SQL dentro de los contenedores Docker que alojan los distintos SGBD. En este entorno, los servidores de bases de datos se ejecutan dentro de contenedores aislados, mientras que los scripts SQL se encuentran almacenados en la máquina virtual anfitriona. Como consecuencia, no siempre es posible acceder directamente a dichos scripts desde el interior del contenedor.

En algunos casos, como en Oracle o PostgreSQL, fue posible ejecutar los scripts redirigiendo la entrada estándar del cliente SQL hacia el contenedor mediante el comando docker exec. Sin embargo, en otros casos, como en IBM DB2, este enfoque no funcionó. Por ello fue necesario copiar previamente los scripts al contenedor utilizando docker cp y ejecutarlos posteriormente desde una ruta temporal dentro del propio contenedor.

Esta situación muestra la necesidad de considerar la interacción entre el sistema anfitrión y los contenedores al ejecutar herramientas de línea de comandos en entornos aislados.

10.4. Limitaciones del cliente SQL*Plus y del uso del usuario SYS

En el fichero de creación de la base de datos Oracle saltó un error al ejecutar el fichero porque había un salto de línea en la creación de una tabla (para poner un comentario y que

se viese la separación) y tras él un CHECK, pero la consola utilizada es muy antigua y tiene una regla muy estricta: Si dejas una línea totalmente en blanco dentro de un CREATE TABLE, SQL*Plus cree que has terminado de escribir el comando. Se solucionó borrando dicho salto de línea. Los errores mostrados eran del tipo SP2-0734 y SP2-0042 y son exclusivos de la consola sqlplus.

Al ejecutar de nuevo el fichero de creación apareció el siguiente error: ORA-04089: cannot create triggers on objects owned by SYS debido a que el proceso de ejecución del fichero se hizo con el superusuario de Oracle SYS. Oracle prohíbe tajantemente que se creen triggers en cualquier tabla que pertenezca al usuario SYS. Esto lo hacen para evitar que un trigger mal programado o en bucle rompa la base de datos por completo. Para solucionarlo lo que hicimos fue emplear el usuario SYSTEM que es un administrador con permisos para hacer de todo, pero es un esquema pensado para tareas operativas y sí permite crear triggers.

10.5. Manejo de errores esperados en los scripts de pruebas

Los scripts de pruebas desarrollados para la práctica no solo incluyen operaciones correctas, sino también operaciones incorrectas cuyo objetivo es verificar el funcionamiento de las restricciones de integridad y de la lógica de negocio implementada en la base de datos. Por ejemplo, se incluyen pruebas que intentan insertar clientes menores de edad, crear cuentas con saldo negativo o realizar operaciones que dejarían una cuenta con saldo insuficiente. Estas operaciones generan errores en el SGBD, pero dichos errores forman parte del comportamiento esperado del sistema.

Durante la práctica fue necesario distinguir entre errores reales, que indican un problema en la implementación del esquema, y errores esperados que confirman que las restricciones definidas están funcionando correctamente. Esta situación muestra que la interpretación de los mensajes de error de un SGBD requiere considerar el contexto en el que se produce cada operación, ya que un error no siempre implica necesariamente un fallo en el sistema.

10.6. Reproducibilidad y automatización del proceso

Se observó la importancia de la reproducibilidad del trabajo realizado. No basta con que un conjunto de scripts funcione correctamente en una ejecución. Para que sea reutilizable, los scripts deben poder ejecutarse repetidamente en distintos momentos y producir siempre el mismo resultado. Para ello fue necesario diseñar scripts reejecutables que eliminen previamente los objetos existentes, estructurar adecuadamente los directorios del proyecto y automatizar, cuando sea posible, la ejecución de los scripts SQL. Además, la utilización de contenedores Docker contribuye a reforzar esta reproducibilidad, ya que permite disponer de entornos de ejecución controlados e independientes del sistema anfitrión.

11. Organización del trabajo y esfuerzos invertidos

Apartados del trabajo realizados	Lucía	Luna
1. Diseño conceptual y lógico de la base de datos		
Diseño conceptual	2	2
Diseño lógico relacional	2	1
Implementación modelo relacional	2,5	0,5
Diseño modelo lógico/relacional	3,5	1
2. Implementación de sistemas objeto-relacionales (SGBD)		
Implementación modelo lógico/relacional Oracle	2	0
Implementación modelo lógico/relacional PostgreSQL	1	1
Implementación modelo lógico/relacional DB2	0	3
3. Implementación de base de datos orientada a objetos pura		
Implementación con DB4o	2,5	0,5
4. Investigación, validación y documentación		
Recopilación de dificultades encontradas y soluciones	1,5	2
Redacción, formato general de la memoria y revisión final	3	9
Preparación de <i>scripts</i>	6	6
	26	26

12. Bibliografía

- db4objects Inc.** (2013). *db4o 7.10 Tutorial - Java*. [ODBMS.org](https://www.odbms.org/wp-content/uploads/2013/11/db4o-7.10-tutorial-java.pdf).
<https://www.odbms.org/wp-content/uploads/2013/11/db4o-7.10-tutorial-java.pdf>
- IBM.** (2026, 9 de enero). *Db2 for Linux, UNIX and Windows*. IBM Documentation.
<https://www.ibm.com/docs/es/db2/11.5.x?topic=objects-user-defined-types>
- Ilarri Artigas, S.** (2026, 6 de febrero). *Práctica 2: Diseño de Bases de Datos con Características de Orientación a Objetos* [Documento PDF]. Moodle. Universidad de Zaragoza.
- Oracle.** (2019, enero). *Object-Relational Developer's Guide*. Oracle Help Center.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/adobj/index.html>
- PostgreSQL Global Development Group.** (2026, 26 de febrero). 5.11. Inheritance. PostgreSQL Documentation.
<https://www.postgresql.org/docs/current/ddl-inherit.html>

13. Anexos

13.1. Anexo I. Ejecución manual de los scripts en los distintos SGBD

Los scripts y configuraciones del proyecto se organizan, opcionalmente, siguiendo una estructura de directorios que separa los scripts SQL de los ficheros de configuración de los contenedores Docker. Esta organización no es estrictamente necesaria para la ejecución de los scripts, pero facilita la gestión del proyecto y permite distinguir claramente entre los distintos componentes utilizados en la práctica. La estructura relevante es la siguiente:

```
bd2/
├── docker/
│   ├── oracle/
│   │   └── docker-compose.yml
│   ├── postgresql/
│   │   └── docker-compose.yml
│   └── db2/
│       └── docker-compose.yml
└── sql/
    ├── oracle/
    │   ├── bd-bancaria-oracle.sql
    │   ├── bd-bancaria-pruebas-oracle.sql
    │   ├── bd-bancaria-or-oracle.sql
    │   └── bd-bancaria-or-pruebas-oracle.sql
    ├── postgresql/
    │   ├── bd-bancaria-or-postgresql.sql
    │   └── bd-bancaria-or-pruebas-postgresql.sql
    └── db2/
        ├── bd-bancaria-or-db2.sql
        └── bd-bancaria-or-pruebas-db2.sql
```

En esta estructura:

- el directorio `docker/` contiene la configuración necesaria para ejecutar cada sistema gestor mediante contenedores Docker,
- y el directorio `sql/` contiene los scripts SQL organizados por SGBD.

A partir de esta organización, la ejecución manual de los scripts se realiza invocando los clientes SQL de cada gestor dentro de sus respectivos contenedores Docker.

13.1.1. Oracle

El gestor Oracle se ejecuta en un contenedor Docker denominado: oracle-xe. La conexión al sistema se realiza mediante sqlplus, utilizando el usuario administrativo SYSTEM.

Ejecución del script de creación del esquema relacional

El script se ejecutó desde la máquina virtual utilizando redirección de entrada estándar hacia el cliente sqlplus del contenedor:

```
docker exec -i oracle-xe sqlplus system/oracle123@localhost/XEPDB1 <
~/bd2/sql/oracle/bd-bancaria-oracle.sql
```

Ejecución del script de pruebas del modelo relacional

Posteriormente se ejecutó el script de pruebas funcionales:

```
docker exec -i oracle-xe sqlplus system/oracle123@localhost/XEPDB1 <
~/bd2/sql/oracle/bd-bancaria-pruebas-oracle.sql
```

Ejecución del script de creación del esquema objeto-relacional

A continuación se ejecutó el script que implementa el modelo objeto-relacional en Oracle:

```
docker exec -i oracle-xe sqlplus system/oracle123@localhost/XEPDB1 <
~/bd2/sql/oracle/bd-bancaria-or-oracle.sql
```

Este script define los tipos objeto, las colecciones y las tablas tipadas necesarias para representar el modelo objeto-relacional.

Ejecución del script de pruebas del modelo objeto-relacional

Finalmente se ejecutó el script de pruebas correspondiente al modelo objeto-relacional:

```
docker exec -i oracle-xe sqlplus system/oracle123@localhost/XEPDB1 <
~/bd2/sql/oracle/bd-bancaria-or-pruebas-oracle.sql
```

Este script inserta datos de prueba y ejecuta diversas operaciones destinadas a verificar el funcionamiento de las restricciones y de la lógica implementada en el esquema objeto-relacional.

13.1.2. PostgreSQL

El sistema PostgreSQL se ejecuta en el contenedor: postgres-practica1. La conexión se realiza mediante el cliente psql utilizando el usuario alumno.

Ejecución del script de creación del esquema

```
docker exec -i -e PGPASSWORD=alumno123 postgres-practica1 psql -U alumno -d
postgres < ~/bd2/sql/postgresql/bd-bancaria-or-postgresql.sql
```

Ejecución del script de pruebas

El script de pruebas se ejecutó mediante:

```
docker exec -i -e PGPASSWORD=alumno123 postgres-practica1 psql -U alumno -d
postgres < ~/bd2/sql/postgresql/bd-bancaria-or-pruebas-postgresql.sql
```

13.1.3. IBM DB2

El gestor IBM DB2 se ejecuta en el contenedor: contenedor_db2. Los scripts se ejecutan mediante el usuario de instancia: db2inst1. Debido a que el cliente db2 no permite ejecutar directamente scripts ubicados en el sistema de archivos del host, los ficheros SQL se copiaron previamente al contenedor.

Copia de scripts al contenedor

```
docker cp ~/bd2/sql/db2/bd-bancaria-or-db2.sql contenedor_db2:/tmp/
docker cp ~/bd2/sql/db2/bd-bancaria-or-pruebas-db2.sql contenedor_db2:/tmp/
```

Ejecución del script de creación del esquema

```
docker exec -it contenedor_db2 su - db2inst1 -c "db2 connect to DB2INST1 && db2
-t@ -vf /tmp/bd-bancaria-or-db2.sql"
```

Ejecución del script de pruebas

```
docker exec -it contenedor_db2 su - db2inst1 -c "db2 connect to DB2INST1 && db2
-t@ -vf /tmp/bd-bancaria-or-pruebas-db2.sql"
```

13.2. Anexo II. Scripts SQL del modelo relacional

13.2.1. bd-bancaria-oracle.sql

```
-- =====
-- Script: bd-bancaria-oracle.sql
-- SGBD: Oracle
-- Modelo: Relacional
-- Proposito: Creacion del esquema relacional de la base de datos bancaria
-- Requiere: Ninguno
-- Observaciones: El script elimina previamente tablas, secuencia,
-- procedimiento y job scheduler si ya existen.
-- =====
```

```
BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Operacion CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
```

```
BEGIN
```

```
EXECUTE IMMEDIATE 'DROP TABLE ser_titular CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Cliente CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Cuenta CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Oficina CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP SEQUENCE seq_operacion';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP PROCEDURE liquidar_intereses';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
  DBMS_SCHEDULER.DROP_JOB('JOB_LIQUIDAR_INTERESES', force => TRUE);
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

CREATE TABLE Cliente
(
  Dni          VARCHAR2(9) PRIMARY KEY,
  Nombre       VARCHAR2(50) NOT NULL,
  Email        VARCHAR2(50) CHECK (Email LIKE '%@%.%'),
  Apellidos    VARCHAR2(50) NOT NULL,
  Edad         NUMBER(3)    NOT NULL CHECK (Edad >= 18),
  Direccion    VARCHAR2(100) NOT NULL,
  Telefono     NUMBER(9)    NOT NULL,
  CHECK (REGEXP_LIKE(Dni, '^[0-9]{8}[a-zA-Z]$'))
);

CREATE TABLE Oficina
(
  Codigo       NUMBER(5) PRIMARY KEY,
  Telefono     NUMBER(9)    NOT NULL,
```

```

    Direccion    VARCHAR2(100) NOT NULL
);

CREATE TABLE Cuenta
(
    Iban          VARCHAR2(24) PRIMARY KEY,
    Numero        NUMBER(20) NOT NULL,
    Fecha_creacion DATE DEFAULT SYSDATE NOT NULL,
    Saldo_actual  NUMBER(10,2) NOT NULL CHECK (Saldo_actual >= 0),
    Es_ahorro     NUMBER(1) NOT NULL CHECK (Es_ahorro IN (0,1)),
    Tipo_interes  NUMBER(5,2),
    Oficina_codigo NUMBER(5),
    FOREIGN KEY (Oficina_codigo) REFERENCES Oficina(Codigo),
    CHECK (
        (Es_ahorro = 1 AND Tipo_interes IS NOT NULL AND Oficina_codigo IS NULL)
        OR
        (Es_ahorro = 0 AND Tipo_interes IS NULL AND Oficina_codigo IS NOT NULL)
    )
);

CREATE TABLE Operacion
(
    Codigo          NUMBER(8),
    Concepto        VARCHAR2(250),
    Fecha           DATE DEFAULT SYSDATE NOT NULL,
    Cantidad        NUMBER(10,2) NOT NULL CHECK (Cantidad > 0),
    Tipo_operacion  VARCHAR2(13) NOT NULL CHECK (Tipo_operacion IN ('Ingreso', 'Retirada',
'Transferencia')),
    Cuenta_origen   VARCHAR2(24) NOT NULL,
    Oficina_codigo NUMBER(5),
    Cuenta_destino  VARCHAR2(24),
    PRIMARY KEY (Codigo, Cuenta_origen),
    FOREIGN KEY (Cuenta_origen) REFERENCES Cuenta(Iban),
    FOREIGN KEY (Oficina_codigo) REFERENCES Oficina(Codigo),
    FOREIGN KEY (Cuenta_destino) REFERENCES Cuenta(Iban),
    CHECK (Cuenta_origen != Cuenta_destino),
    CHECK (
        (Tipo_operacion = 'Transferencia' AND Cuenta_destino IS NOT NULL AND
Oficina_codigo IS NULL)
        OR
        (Tipo_operacion IN ('Ingreso', 'Retirada') AND Oficina_codigo IS NOT NULL AND
Cuenta_destino IS NULL)
        OR
        (Tipo_operacion = 'Ingreso' AND Concepto = 'Liquidacion de intereses mensuales'
AND Oficina_codigo IS NULL AND Cuenta_destino IS NULL)
    )
);

CREATE TABLE ser_titular
(
    Cliente_dni    VARCHAR2(9),
    Cuenta_iban    VARCHAR2(24),
    PRIMARY KEY (Cliente_dni, Cuenta_iban),
    FOREIGN KEY (Cliente_dni) REFERENCES Cliente(Dni),
    FOREIGN KEY (Cuenta_iban) REFERENCES Cuenta(Iban)
);

```

```

CREATE OR REPLACE TRIGGER trg_valida_fecha_operacion
BEFORE INSERT OR UPDATE ON Operacion
FOR EACH ROW
DECLARE
    v_fecha_creacion_origen DATE;
    v_fecha_creacion_destino DATE;
BEGIN
    SELECT Fecha_creacion INTO v_fecha_creacion_origen
    FROM Cuenta WHERE Iban = :NEW.Cuenta_origen;

    IF :NEW.Fecha < v_fecha_creacion_origen THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: La operación es anterior a la creación de
la cuenta origen.');
```

la cuenta origen.');

```

    END IF;

    IF :NEW.Tipo_operacion = 'Transferencia' AND :NEW.Cuenta_destino IS NOT NULL THEN
        SELECT Fecha_creacion INTO v_fecha_creacion_destino
        FROM Cuenta WHERE Iban = :NEW.Cuenta_destino;

        IF :NEW.Fecha < v_fecha_creacion_destino THEN
            RAISE_APPLICATION_ERROR(-20002, 'Error: La operación es anterior a la
creación de la cuenta destino.');
```

creación de la cuenta destino.');

```

        END IF;
    END IF;
END;
/

CREATE OR REPLACE TRIGGER trg_actualiza_saldo
AFTER INSERT ON Operacion
FOR EACH ROW
BEGIN
    IF :NEW.Tipo_operacion = 'Ingreso' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;

    ELSIF :NEW.Tipo_operacion = 'Retirada' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;

    ELSIF :NEW.Tipo_operacion = 'Transferencia' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_destino;
    END IF;
END;
/

CREATE SEQUENCE seq_operacion START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE PROCEDURE liquidar_intereses AS
    v_interes_calculado NUMBER(10,2);
BEGIN
    FOR cuenta_rec IN (SELECT Iban, Saldo_actual, Tipo_interes, Oficina_codigo
                        FROM Cuenta
                        WHERE Es_ahorro = 1 AND Tipo_interes IS NOT NULL AND Tipo_interes
> 0)

```

```

    LOOP
        v_interes_calculado := cuenta_rec.Saldo_actual * (cuenta_rec.Tipo_interes / 100 /
12);

        IF v_interes_calculado > 0 THEN
            INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion,
Cuenta_origen, Oficina_codigo, Cuenta_destino)
                VALUES (seq_operacion.NEXTVAL, 'Liquidacion de intereses mensuales', SYSDATE,
v_interes_calculado, 'Ingreso', cuenta_rec.Iban, cuenta_rec.Oficina_codigo, NULL);
            END IF;
        END LOOP;

        COMMIT;
    END;
/

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_LIQUIDAR_INTERESES',
        job_type           => 'PLSQL_BLOCK',
        job_action         => 'BEGIN liquidar_intereses; END;',
        start_date         => SYSTIMESTAMP,
        repeat_interval    => 'FREQ=DAILY; BYHOUR=2; BYMINUTE=0; BYSECOND=0',
        enabled            => TRUE,
        comments           => 'Job para calcular y pagar intereses de cuentas de ahorro cada
madrugada'
    );
END;
/

```

13.2.2. bd-bancaria-pruebas-oracle.sql

```

-- =====
-- Script: bd-bancaria-pruebas-oracle.sql
-- SGBD: Oracle
-- Modelo: Relacional
-- Proposito: Insercion de datos y pruebas funcionales del modelo
-- Requiere: bd-bancaria-oracle.sql
-- Observaciones:
--   - Inserta datos base para clientes, cuentas y oficinas.
--   - Ejecuta pruebas de restricciones (algunos ORA- son esperados).
--   - Verifica triggers de operaciones y actualizacion de saldo.
--   - Comprueba el funcionamiento del procedimiento de liquidacion
--     de intereses.
-- =====

-- Configuracion de formato para mejorar la visualizacion en SQL*Plus
SET LINESIZE 150;
SET PAGESIZE 50;
ALTER SESSION SET NLS_DATE_FORMAT = 'DD/MM/YYYY HH24:MI:SS';

PROMPT =====;
PROMPT 0. LIMPIEZA DE DATOS PREVIOS;
PROMPT =====;

-- Borramos en orden para respetar las claves foraneas (hijos primero, luego padres)
DELETE FROM ser_titular;

```

```

DELETE FROM Operacion;
DELETE FROM Cuenta;
DELETE FROM Cliente;
DELETE FROM Oficina;
COMMIT;
PROMPT Datos anteriores eliminados correctamente.

PROMPT =====;
PROMPT 1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE);
PROMPT =====;

-- Insertar Oficinas
INSERT INTO Oficina (Codigo, Telefono, Direccion) VALUES (1, 976111222, 'Paseo
Independencia, Zaragoza');
INSERT INTO Oficina (Codigo, Telefono, Direccion) VALUES (2, 911222333, 'Gran Vía,
Madrid');

-- Insertar Clientes
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
VALUES ('12345678A', 'Ana', 'García', 'ana@email.com', 25, 'Calle Mayor 1', 600111222);
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
VALUES ('87654321B', 'Luis', 'Pérez', 'luis@email.com', 40, 'Calle Menor 2', 600333444);

-- Insertar Cuentas (Nacen con Saldo_actual a 0)
-- Cuenta Corriente para Ana
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
VALUES ('ES9100001111222233334444', 1111222233334444, 0, NULL, 1, 0);

-- Cuenta de Ahorro para Luis
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
VALUES ('ES9100005555666677778888', 5555666677778888, 1, 2.5, NULL, 0);

-- Asignar titulares
INSERT INTO ser_titular (Cliente_dni, Cuenta_iban) VALUES ('12345678A',
'ES9100001111222233334444');
INSERT INTO ser_titular (Cliente_dni, Cuenta_iban) VALUES ('87654321B',
'ES9100005555666677778888');

COMMIT;
PROMPT Datos base insertados.

PROMPT =====;
PROMPT 2. PRUEBAS DE RESTRICCIONES (AQUI DEBEN SALIR ERRORES ORA-);
PROMPT =====;

PROMPT > Prueba 2.1: Intentar insertar un DNI falso (Falla por Regex)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
VALUES ('123A45678', 'Falso', 'Dni', 'falso@email.com', 30, 'Dir', 600000000);

PROMPT > Prueba 2.2: Intentar insertar un menor de edad (Falla por CHECK)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
VALUES ('11111111C', 'Joven', 'Menor', 'joven@email.com', 16, 'Dir', 600000000);

PROMPT > Prueba 2.3: Cuenta de Ahorro sin tipo de interes (Falla por CHECK de jerarquia)
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
VALUES ('ES9100009999000011112222', 9999000011112222, 1, NULL, NULL, 0);

```

```

PROMPT > Prueba 2.4: Transferencia sin Cuenta Destino (Falla por CHECK de operacion)
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion, Cuenta_origen,
Oficina_codigo, Cuenta_destino)
VALUES (seq_operacion.NEXTVAL, 'Transf fallida', 100, 'Transferencia',
'ES91000011112223334444', NULL, NULL);

PROMPT =====;
PROMPT 3. PRUEBAS DE TRIGGERS Y OPERACIONES (DEBEN FUNCIONAR);
PROMPT =====;

PROMPT > Hacemos un Ingreso de 1000 euros en la cuenta de Ana
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion, Cuenta_origen,
Oficina_codigo, Cuenta_destino)
VALUES (seq_operacion.NEXTVAL, 'Nomina', 1000, 'Ingreso', 'ES91000011112223334444', 1,
NULL);

PROMPT > Hacemos una Retirada de 200 euros en la cuenta de Ana
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion, Cuenta_origen,
Oficina_codigo, Cuenta_destino)
VALUES (seq_operacion.NEXTVAL, 'Cajero', 200, 'Retirada', 'ES91000011112223334444', 1,
NULL);

PROMPT > Ana hace una transferencia de 300 euros a Luis
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion, Cuenta_origen,
Oficina_codigo, Cuenta_destino)
VALUES (seq_operacion.NEXTVAL, 'Regalo', 300, 'Transferencia',
'ES91000011112223334444', NULL, 'ES9100005555666677778888');

PROMPT > ESTADO DE SALDOS ESPERADO:
PROMPT > Ana empezo con 0 + 1000 - 200 - 300 = 500 euros.
PROMPT > Luis empezo con 0 + 300 transferidos = 300 euros.
SELECT Iban, Saldo_actual FROM Cuenta;

PROMPT =====;
PROMPT 4. PRUEBA DE DISPARADOR EN CADENA (FONDOS INSUFICIENTES);
PROMPT =====;

PROMPT > Ana intenta sacar 600 euros pero solo tiene 500.
PROMPT > El trigger restara el saldo y Oracle parara la insercion por el CHECK de saldo
>= 0.
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion, Cuenta_origen,
Oficina_codigo, Cuenta_destino)
VALUES (seq_operacion.NEXTVAL, 'Compra TV', 600, 'Retirada', 'ES91000011112223334444',
1, NULL);

PROMPT > Comprobamos que el saldo de Ana sigue intacto en 500 euros
SELECT Iban, Saldo_actual FROM Cuenta WHERE Iban = 'ES91000011112223334444';

PROMPT =====;
PROMPT 5. PRUEBA DEL PROCEDIMIENTO DE INTERESES;
PROMPT =====;

PROMPT > Ejecutamos el pago de intereses manual (Liquidacion de la cuenta de ahorro de
Luis)
EXEC liquidar_intereses;

PROMPT > Comprobamos las operaciones. Deberia aparecer un nuevo Ingreso automatico para

```



```

Luis
COLUMN Concepto FORMAT A35;
SELECT Codigo, Concepto, Cantidad, Tipo_operacion FROM Operacion ORDER BY Codigo;

PROMPT > Comprobamos que el saldo de Luis ha subido (Tenia 300, y se le ha sumado su
interes)
SELECT Iban, Saldo_actual FROM Cuenta WHERE Iban = 'ES9100005555666677778888';

COMMIT;
PROMPT PRUEBAS FINALIZADAS.

```

13.3. Anexo III. Scripts SQL del modelo objeto-relacional

13.3.1. bd-bancaria-or-oracle.sql

```

-- =====
-- Script: bd-bancaria-or-oracle.sql
-- SGBD: Oracle
-- Modelo: Objeto-relacional
-- Proposito: Creacion del esquema objeto-relacional de la base
-- Requiere: Ninguno
-- Observaciones:
--   - El script elimina previamente tablas, tipos y jobs existentes.
--   - Define tipos objeto (UDT), jerarquias de tipos y colecciones.
--   - Crea tablas tipadas y nested tables.
--   - Implementa triggers de logica de negocio sobre operaciones.
--   - Incluye un procedimiento y un job scheduler para calcular
--     intereses periodicos de las cuentas de ahorro.
-- =====

-- =====
-- 1. LIMPIEZA DE ENTORNO (DROP)
-- =====

BEGIN
  -- 1. Borramos el JOB si ya existía del intento anterior
  BEGIN
    DBMS_SCHEDULER.DROP_JOB('JOB_INTERESES_DIARIOS', force => TRUE);
  EXCEPTION WHEN OTHERS THEN NULL;
  END;

  -- 2. Borramos las tablas una a una forzando la caída (Mejor que con un bucle FOR)
  BEGIN EXECUTE IMMEDIATE 'DROP TABLE Operacion CASCADE CONSTRAINTS PURGE'; EXCEPTION
WHEN OTHERS THEN NULL; END;
  BEGIN EXECUTE IMMEDIATE 'DROP TABLE Cuenta CASCADE CONSTRAINTS PURGE'; EXCEPTION WHEN
OTHERS THEN NULL; END;
  BEGIN EXECUTE IMMEDIATE 'DROP TABLE Cliente CASCADE CONSTRAINTS PURGE'; EXCEPTION
WHEN OTHERS THEN NULL; END;
  BEGIN EXECUTE IMMEDIATE 'DROP TABLE Oficina CASCADE CONSTRAINTS PURGE'; EXCEPTION
WHEN OTHERS THEN NULL; END;

  -- 3. Borramos los tipos (FORCE para romper dependencias)
  FOR obj IN (SELECT object_name FROM user_objects WHERE object_type = 'TYPE' AND
object_name IN

('LISTAREFSCUENTAS', 'LISTAREFSCLIENTES', 'LISTAREFSCORRIENTES', 'LISTAREFSCUENTAS_CORR',

```

```

'LISTAREFSEFECTIVO','OFICINAUDT','CLIENTEUDT','CUENTAUDT','AHORROUDT','CORRIENTEUDT',
  'OPERACIONUDT','TRANSFERENCIAUDT','EFECTIVOUDT')) LOOP
  EXECUTE IMMEDIATE 'DROP TYPE ' || obj.object_name || ' FORCE';
END LOOP;
END;
/

-- =====
-- 2. DEFINICIÓN DE TIPOS (UDT) Y COLECCIONES
-- =====

-- Declaraciones anticipadas para evitar errores de compilación
CREATE OR REPLACE TYPE CuentaUdt;
/
CREATE OR REPLACE TYPE ClienteUdt;
/
CREATE OR REPLACE TYPE OficinaUdt;
/
CREATE OR REPLACE TYPE CorrienteUdt;
/
CREATE OR REPLACE TYPE EfectivoUdt;
/

-- Colecciones para Las Nested Tables
CREATE OR REPLACE TYPE ListaRefsCuentas AS TABLE of ref CuentaUdt;
/
CREATE OR REPLACE TYPE ListaRefsClientes AS TABLE of ref ClienteUdt;
/
CREATE OR REPLACE TYPE ListaRefsCorrientes AS TABLE of ref CorrienteUdt;
/
CREATE OR REPLACE TYPE ListaRefsEfectivo AS TABLE of ref EfectivoUdt;
/

-- Tipo Cliente
CREATE OR REPLACE TYPE ClienteUdt AS OBJECT (
  Dni          VARCHAR2(9),
  Nombre       VARCHAR2(50),
  Email        VARCHAR2(50),
  Apellidos    VARCHAR2(50),
  Edad         NUMBER(3),
  Direccion    VARCHAR2(100),
  Telefono     NUMBER(9),
  refCuenta    ListaRefsCuentas
) NOT FINAL;
/

-- Tipo Cuenta y Subtipos
CREATE OR REPLACE TYPE CuentaUdt AS OBJECT (
  Iban          VARCHAR2(24),
  Numero        NUMBER(20),
  Fecha_creacion DATE,
  Saldo_actual  NUMBER(10,2),
  refCliente    ListaRefsClientes
) NOT FINAL;
/

CREATE OR REPLACE TYPE AhorroUdt under CuentaUdt (

```

```

        Tipo_interes    NUMBER(10,2)
    );
/

CREATE OR REPLACE TYPE CorrienteUdt under CuentaUdt (
    refOficina          REF OficinaUdt
);
/

-- Tipo Oficina (con Las colecciones de referencias)
CREATE OR REPLACE TYPE OficinaUdt AS OBJECT (
    Codigo              NUMBER(5),
    Telefono            NUMBER(9),
    Direccion           VARCHAR2(100),
    refCorriente        ListaRefsCorrientes,
    refEfectivo         ListaRefsEfectivo
) NOT FINAL;
/

-- Tipo operación y subtipos
CREATE OR REPLACE TYPE OperacionUdt AS OBJECT (
    Codigo              NUMBER(8),
    Concepto            VARCHAR2(250),
    Fecha              TIMESTAMP,
    Cantidad            NUMBER(10,2),
    refCuentaOrigen     REF CuentaUdt
) NOT FINAL;
/

CREATE OR REPLACE TYPE TransferenciaUdt under OperacionUdt (
    refCuentaDestino     REF CuentaUdt
);
/

CREATE OR REPLACE TYPE EfectivoUdt under OperacionUdt (
    refOficina          REF OficinaUdt
);
/

-- =====
-- 3. CREACIÓN DE TABLAS TIPADAS
-- =====

CREATE TABLE Oficina of OficinaUdt (
    PRIMARY KEY (Codigo)
) NESTED TABLE refCorriente STORE AS tab_ref_corr,
   NESTED TABLE refEfectivo STORE AS tab_ref_efec;

CREATE TABLE Cliente of ClienteUdt (
    PRIMARY KEY (Dni),
    CONSTRAINT cli_edad_val CHECK (Edad >= 18),
    CONSTRAINT cli_email_chk CHECK (Email LIKE '%@%.%'),
    CONSTRAINT cli_dni_chk CHECK (REGEXP_LIKE(Dni, '^[0-9]{8}[a-zA-Z]$'))
) NESTED TABLE refCuenta STORE AS tab_ref_cue_cli;

CREATE TABLE Cuenta of CuentaUdt (
    PRIMARY KEY (Iban),

```

```

        CONSTRAINT cue_saldo_chk CHECK (Saldo_actual >= 0)
    ) NESTED TABLE refCliente STORE AS tab_ref_cli_cue;

CREATE TABLE Operacion OF OperacionUdt (
    PRIMARY KEY (Codigo)
);

-- =====
-- 4. TRIGGERS DE INTEGRIDAD Y LÓGICA DE NEGOCIO
-- =====

CREATE OR REPLACE TRIGGER TRG_OPERACIONES_COMPUESTO
FOR INSERT ON Operacion
COMPOUND TRIGGER

    -- 1. Declaramos una estructura en memoria para guardar temporalmente las operaciones
    TYPE r_operacion IS RECORD (
        Codigo NUMBER,
        Cantidad NUMBER,
        Fecha TIMESTAMP
    );
    TYPE t_operaciones IS TABLE OF r_operacion;
    v_operaciones t_operaciones := t_operaciones();

    -- FASE A: Se ejecuta fila a fila durante la inserción
    BEFORE EACH ROW IS
        v_fecha_apertura_orig DATE;
    BEGIN
        -- Validar fecha cuenta origen
        SELECT Fecha_creacion INTO v_fecha_apertura_orig
        FROM Cuenta c WHERE REF(c) = :NEW.refCuentaOrigen;

        IF CAST(:NEW.Fecha AS DATE) < v_fecha_apertura_orig THEN
            RAISE_APPLICATION_ERROR(-20002, 'Fecha operación anterior a apertura cuenta
origen.');
```

origen.');

```

        END IF;

        -- Restar saldo en la cuenta origen inmediatamente
        UPDATE Cuenta c SET c.Saldo_actual = c.Saldo_actual - :NEW.Cantidad
        WHERE REF(c) = :NEW.refCuentaOrigen;

        -- Guardamos los datos en memoria para procesar el destino luego
        v_operaciones.EXTEND;
        v_operaciones(v_operaciones.LAST).Codigo := :NEW.Codigo;
        v_operaciones(v_operaciones.LAST).Cantidad := :NEW.Cantidad;
        v_operaciones(v_operaciones.LAST).Fecha := :NEW.Fecha;
    END BEFORE EACH ROW;

    -- FASE B: Se ejecuta una sola vez al terminar el INSERT (La tabla ya no muta)
    AFTER STATEMENT IS
        v_ref_dest REF CuentaUdt;
        v_fecha_apertura_dest DATE;
    BEGIN
        FOR i IN 1 .. v_operaciones.COUNT LOOP
            BEGIN
                -- Ahora podemos hacer SELECT en Operacion porque ya ha terminado inserción
                SELECT TREAT(VALUE(o) AS TransferenciaUdt).refCuentaDestino
```

```

        INTO v_ref_dest
        FROM Operacion o
        WHERE o.Codigo = v_operaciones(i).Codigo
              AND VALUE(o) IS OF (TransferenciaUdt);

        IF v_ref_dest IS NOT NULL THEN
            -- Validar La fecha de La cuenta destino
            SELECT Fecha_creacion INTO v_fecha_apertura_dest
            FROM Cuenta c WHERE REF(c) = v_ref_dest;

            IF CAST(v_operaciones(i).Fecha AS DATE) < v_fecha_apertura_dest THEN
                RAISE_APPLICATION_ERROR(-20003, 'Fecha transferencia anterior a
apertura cuenta destino.');
```

aperturas cuentas destino.');

```

            END IF;

            -- Sumar el saldo en La cuenta destino
            UPDATE Cuenta c SET c.Saldo_actual = c.Saldo_actual +
v_operaciones(i).Cantidad
            WHERE REF(c) = v_ref_dest;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            NULL; -- Si error, es que era EfectivoUdt (no transferencia), ignoramos.
    END;
END LOOP;
END AFTER STATEMENT;

END TRG_OPERACIONES_COMPUUESTO;
/

-- =====
-- 5. CÁLCULO DE INTERESES (PROCEDIMIENTO Y JOB 02:00 AM)
-- =====

-- Procedimiento que recorre las cuentas de ahorro y aplica el interés
CREATE OR REPLACE PROCEDURE PROC_INTERESES_NOCTURNOS IS
BEGIN
    UPDATE Cuenta c
    SET c.Saldo_actual = c.Saldo_actual + (c.Saldo_actual * (TREAT(VALUE(c) AS
AhorroUdt).Tipo_interes / 100))
    WHERE VALUE(c) IS OF (AhorroUdt);
    COMMIT;
END;
/

-- Programación del Job para Las 2 de La mañana
BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_INTERESES_DIARIOS',
        job_type           => 'STORED_PROCEDURE',
        job_action         => 'PROC_INTERESES_NOCTURNOS',
        start_date         => SYSTIMESTAMP,
        repeat_interval    => 'FREQ=DAILY; BYHOUR=2; BYMINUTE=0; BYSECOND=0',
        enabled            => TRUE
    );
END;
/

```

13.3.2. bd-bancaria-or-postgresql.sql

```
-- =====
-- Script: bd-bancaria-or-postgresql.sql
-- SGBD: PostgreSQL
-- Modelo: Objeto-relacional
-- Proposito: Creacion del esquema objeto-relacional en PostgreSQL
-- Requiere: Ninguno
-- Observaciones:
--   - El script elimina previamente todas las tablas mediante DROP CASCADE.
--   - Implementa jerarquias de tablas utilizando INHERITS.
--   - Sustituye las Nested Tables de Oracle por arrays nativos de PostgreSQL.
--   - Define funciones y triggers en PL/pgSQL para implementar la logica
--     de negocio de las operaciones bancarias.
--   - Incluye un procedimiento para calcular los intereses de las
--     cuentas de ahorro.
-- =====

-- =====
-- 1. LIMPIEZA DE ENTORNO
-- =====
-- En PostgreSQL el CASCADE lo limpia todo (tablas y funciones relacionadas)
DROP TABLE IF EXISTS OperacionEfectivo CASCADE;
DROP TABLE IF EXISTS Transferencia CASCADE;
DROP TABLE IF EXISTS Operacion CASCADE;
DROP TABLE IF EXISTS CuentaCorriente CASCADE;
DROP TABLE IF EXISTS CuentaAhorro CASCADE;
DROP TABLE IF EXISTS Cuenta CASCADE;
DROP TABLE IF EXISTS Cliente CASCADE;
DROP TABLE IF EXISTS Oficina CASCADE;

-- =====
-- 2. CREACIÓN DE TABLAS (CON ARRAYS E INHERITS)
-- =====

CREATE TABLE Oficina (
    Codigo INT PRIMARY KEY,
    Telefono INT NOT NULL,
    Direccion VARCHAR(100) NOT NULL,
    -- En PostgreSQL usamos arrays nativos en lugar de Nested Tables
    refCorriente VARCHAR(24)[],
    refEfectivo INT[]
);

CREATE TABLE Cliente (
    Dni VARCHAR(9) PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Email VARCHAR(50) CHECK (Email LIKE '%@%.%'),
    Apellidos VARCHAR(50) NOT NULL,
    Edad INT CHECK (Edad >= 18),
    Direccion VARCHAR(100) NOT NULL,
    Telefono INT NOT NULL,
    refCuenta VARCHAR(24)[] -- Array de IBANs
);

-- Tabla Padre (Cuenta)
```

```

CREATE TABLE Cuenta (
    Iban VARCHAR(24) PRIMARY KEY,
    Numero NUMERIC(20) NOT NULL,
    Fecha_creacion TIMESTAMP NOT NULL,
    Saldo_actual DECIMAL(10,2) CHECK (Saldo_actual >= 0),
    refCliente VARCHAR(9)[] -- Array de DNIs
);

-- Tablas Hijas de Cuenta (Usan INHERITS)
CREATE TABLE CuentaAhorro (
    Tipo_interes DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (Iban) -- En PostgreSQL hay que reiterar la PK en las hijas
) INHERITS (Cuenta);

CREATE TABLE CuentaCorriente (
    refOficina INT NOT NULL,
    PRIMARY KEY (Iban)
) INHERITS (Cuenta);

-- Tabla Padre (Operacion)
CREATE TABLE Operacion (
    Codigo INT PRIMARY KEY,
    Concepto VARCHAR(250),
    Fecha TIMESTAMP NOT NULL,
    Cantidad DECIMAL(10,2) NOT NULL,
    refCuentaOrigen VARCHAR(24) NOT NULL
);

-- Tablas hijas de Operacion
CREATE TABLE Transferencia (
    refCuentaDestino VARCHAR(24) NOT NULL,
    PRIMARY KEY (Codigo)
) INHERITS (Operacion);

CREATE TABLE OperacionEfectivo (
    refOficina INT NOT NULL,
    PRIMARY KEY (Codigo)
) INHERITS (Operacion);

-- =====
-- 3. TRIGGERS Y LÓGICA DE NEGOCIO
-- =====

-- 3.1. Función para Transferencias (Resta origen, Suma destino, Valida fechas)
CREATE OR REPLACE FUNCTION func_logica_transferencia()
RETURNS TRIGGER AS $$
DECLARE
    v_fecha_orig TIMESTAMP;
    v_fecha_dest TIMESTAMP;
BEGIN
    -- Validar fecha origen
    SELECT Fecha_creacion INTO v_fecha_orig FROM Cuenta WHERE Iban = NEW.refCuentaOrigen;
    IF NEW.Fecha < v_fecha_orig THEN
        RAISE EXCEPTION 'Fecha operación anterior a apertura cuenta origen.';
    END IF;

    -- Validar fecha destino

```

```

        SELECT Fecha_creacion INTO v_fecha_dest FROM Cuenta WHERE Iban =
NEW.refCuentaDestino;
        IF NEW.Fecha < v_fecha_dest THEN
            RAISE EXCEPTION 'Fecha transferencia anterior a apertura cuenta destino.';
        END IF;

        -- Actualizar saldos (por INHERITS al buscar en Cuenta buscará en Ahorro y Corriente)
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - NEW.Cantidad WHERE Iban =
NEW.refCuentaOrigen;
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + NEW.Cantidad WHERE Iban =
NEW.refCuentaDestino;

        RETURN NEW;
    END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_transferencia
BEFORE INSERT ON Transferencia
FOR EACH ROW EXECUTE FUNCTION func_logica_transferencia();

-- 3.2. Función para Operaciones en Efectivo (Resta origen, Valida fecha)
CREATE OR REPLACE FUNCTION func_logica_efectivo()
RETURNS TRIGGER AS $$
DECLARE
    v_fecha_orig TIMESTAMP;
BEGIN
    -- Validar fecha origen
    SELECT Fecha_creacion INTO v_fecha_orig FROM Cuenta WHERE Iban = NEW.refCuentaOrigen;
    IF NEW.Fecha < v_fecha_orig THEN
        RAISE EXCEPTION 'Fecha operación anterior a apertura cuenta origen.';
    END IF;

    -- Actualizar saldo origen
    UPDATE Cuenta SET Saldo_actual = Saldo_actual - NEW.Cantidad WHERE Iban =
NEW.refCuentaOrigen;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_efectivo
BEFORE INSERT ON OperacionEfectivo
FOR EACH ROW EXECUTE FUNCTION func_logica_efectivo();

-- =====
-- 4. CÁLCULO DE INTERESES (Procedimiento)
-- =====
-- PostgreSQL actualiza solo la tabla CuentaAhorro sin tocar el resto.
CREATE OR REPLACE PROCEDURE proc_intereses_nocturnos()
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE CuentaAhorro
    SET Saldo_actual = Saldo_actual + (Saldo_actual * (Tipo_interes / 100));
END;
$$;
```


13.3.3. bd-bancaria-or-db2.sql

```
-- =====
-- Script: bd-bancaria-or-db2.sql
-- SGBD: IBM DB2
-- Modelo: Objeto-relacional
-- Proposito: Creacion del esquema objeto-relacional en DB2
-- Requiere: Ninguno
-- Observaciones:
--   - El script se ejecuta sobre la base de datos PRACTDB2.
--   - Utiliza terminador de sentencias '@' para permitir
--     bloques BEGIN...END.
--   - Elimina previamente jerarquias de tablas y tipos si existen.
--   - Define tipos estructurados (UDT) y jerarquias de tipos.
--   - Implementa tablas tipadas y jerarquias mediante UNDER.
-- =====

-- Conexion a la base de datos donde se creara el esquema
CONNECT TO DB2INST1
@

-----
-- LIMPIEZA
-----

BEGIN
DECLARE CONTINUE HANDLER FOR SQLSTATE '42704' BEGIN END;
DECLARE CONTINUE HANDLER FOR SQLSTATE '42809' BEGIN END; -- Maneja el error si la
jerarquía no existe aún

-- Borrado de jerarquías de tablas tipadas (Sintaxis corregida)
EXECUTE IMMEDIATE 'DROP TABLE HIERARCHY Operacion';
EXECUTE IMMEDIATE 'DROP TABLE HIERARCHY Cuenta';
EXECUTE IMMEDIATE 'DROP TABLE Cliente';
EXECUTE IMMEDIATE 'DROP TABLE Oficina';

-- Borrado de tipos
EXECUTE IMMEDIATE 'DROP TYPE EfectivoUdt';
EXECUTE IMMEDIATE 'DROP TYPE TransferenciaUdt';
EXECUTE IMMEDIATE 'DROP TYPE OperacionUdt';
EXECUTE IMMEDIATE 'DROP TYPE CorrienteUdt';
EXECUTE IMMEDIATE 'DROP TYPE AhorroUdt';
EXECUTE IMMEDIATE 'DROP TYPE CuentaUdt';
EXECUTE IMMEDIATE 'DROP TYPE ClienteUdt';
EXECUTE IMMEDIATE 'DROP TYPE OficinaUdt';
END
@

-----
-- TIPOS OBJETO-RELACIONALES
-----

CREATE TYPE OficinaUdt AS
(
  Codigo DECIMAL(5,0),
  Telefono DECIMAL(9,0),
  Direccion VARCHAR(100)
)
MODE DB2SQL
```

```
INSTANTIABLE NOT FINAL
@

CREATE TYPE ClienteUdt AS
(
    Dni VARCHAR(9),
    Nombre VARCHAR(50),
    Email VARCHAR(50),
    Apellidos VARCHAR(50),
    Edad DECIMAL(3,0),
    Direccion VARCHAR(100),
    Telefono DECIMAL(9,0)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

CREATE TYPE CuentaUdt AS
(
    Iban VARCHAR(24),
    Numero DECIMAL(20,0),
    Fecha_creacion DATE,
    Saldo_actual DECIMAL(10,2)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

CREATE TYPE AhorroUdt UNDER CuentaUdt AS
(
    Tipo_interes DECIMAL(10,2)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

CREATE TYPE CorrienteUdt UNDER CuentaUdt AS
(
    refOficina DECIMAL(5,0)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

CREATE TYPE OperacionUdt AS
(
    Codigo DECIMAL(8,0),
    Concepto VARCHAR(250),
    Fecha TIMESTAMP,
    Cantidad DECIMAL(10,2),
    refCuentaOrigen VARCHAR(24)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

CREATE TYPE TransferenciaUdt UNDER OperacionUdt AS
```

```

(
    refCuentaDestino VARCHAR(24)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

CREATE TYPE EfectivoUdt UNDER OperacionUdt AS
(
    refOficina DECIMAL(5,0)
)
MODE DB2SQL
INSTANTIABLE NOT FINAL
@

-----
-- TABLAS TIPADAS (MODELO OBJETO-RELACIONAL)
-----

-- Tablas sin jerarquía
CREATE TABLE Oficina OF OficinaUdt
(
    REF IS oid_oficina USER GENERATED,
    Codigo WITH OPTIONS NOT NULL PRIMARY KEY,
    Telefono WITH OPTIONS NOT NULL,
    Direccion WITH OPTIONS NOT NULL
)
@

CREATE TABLE Cliente OF ClienteUdt
(
    REF IS oid_cliente USER GENERATED,
    Dni WITH OPTIONS NOT NULL PRIMARY KEY,
    Nombre WITH OPTIONS NOT NULL,
    Apellidos WITH OPTIONS NOT NULL,
    Edad WITH OPTIONS NOT NULL CHECK (Edad >= 18),
    Direccion WITH OPTIONS NOT NULL,
    Telefono WITH OPTIONS NOT NULL
)
@

-- Jerarquía de Cuentas
CREATE TABLE Cuenta OF CuentaUdt
(
    REF IS oid_cuenta USER GENERATED,
    Iban WITH OPTIONS NOT NULL PRIMARY KEY,
    Numero WITH OPTIONS NOT NULL,
    Fecha_creacion WITH OPTIONS NOT NULL,
    Saldo_actual WITH OPTIONS NOT NULL CHECK (Saldo_actual >= 0)
)
@

CREATE TABLE CuentaAhorro OF AhorroUdt UNDER Cuenta
INHERIT SELECT PRIVILEGES
(
    Tipo_interes WITH OPTIONS NOT NULL CHECK (Tipo_interes > 0)
)

```

```

@

CREATE TABLE CuentaCorriente OF CorrienteUdt UNDER Cuenta
INHERIT SELECT PRIVILEGES
(
    refOficina WITH OPTIONS NOT NULL
)
@

-- Jerarquía de Operaciones
CREATE TABLE Operacion OF OperacionUdt
(
    REF IS oid_operacion USER GENERATED,
    Codigo WITH OPTIONS NOT NULL PRIMARY KEY,
    Fecha WITH OPTIONS NOT NULL,
    Cantidad WITH OPTIONS NOT NULL CHECK (Cantidad > 0),
    refCuentaOrigen WITH OPTIONS NOT NULL
)
@

CREATE TABLE Transferencia OF TransferenciaUdt UNDER Operacion
INHERIT SELECT PRIVILEGES
(
    refCuentaDestino WITH OPTIONS NOT NULL
)
@

CREATE TABLE OperacionEfectivo OF EfectivoUdt UNDER Operacion
INHERIT SELECT PRIVILEGES
(
    refOficina WITH OPTIONS NOT NULL
)
@

CONNECT RESET
@

```

13.3.4. bd-bancaria-or-pruebas-oracle.sql

```

-- =====
-- Script: bd-bancaria-or-pruebas-oracle.sql
-- SGBD: Oracle
-- Modelo: Objeto-relacional
-- Proposito: Insercion de datos y pruebas funcionales del
--           modelo objeto-relacional
-- Requiere: bd-bancaria-or-oracle.sql
-- Observaciones:
--   - Inserta datos base utilizando constructores de tipos objeto.
--   - Actualiza relaciones mediante colecciones y referencias (REF).
--   - Ejecuta pruebas negativas de restricciones y triggers
--     (algunos errores ORA- son esperados).
--   - Verifica el funcionamiento de los triggers de operaciones
--     y del procedimiento de calculo de intereses.
-- =====

-- Configuracion de formato para mejorar la visualizacion en SQL*Plus
SET LINESIZE 150;

```

```

SET PAGESIZE 50;
ALTER SESSION SET NLS_DATE_FORMAT = 'DD/MM/YYYY HH24:MI:SS';
PROMPT =====;
PROMPT 0. LIMPIEZA DE DATOS PREVIOS;
PROMPT =====;

-- Borramos en orden para respetar las referencias
DELETE FROM Operacion;
DELETE FROM Cuenta;
DELETE FROM Cliente;
DELETE FROM Oficina;
COMMIT;
PROMPT Datos anteriores eliminados correctamente.

PROMPT =====;
PROMPT 1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE);
PROMPT =====;

-- Insertar Oficinas (Inicializando sus listas de referencias vacías)
INSERT INTO Oficina (Codigo, Telefono, Direccion, refCorriente, refEfectivo)
VALUES (1, 976111222, 'Paseo Independencia, Zaragoza', ListaRefsCorrientes(),
ListaRefsEfectivo());

INSERT INTO Oficina (Codigo, Telefono, Direccion, refCorriente, refEfectivo)
VALUES (2, 911222333, 'Gran Vía, Madrid', ListaRefsCorrientes(), ListaRefsEfectivo());

-- Insertar Clientes (Cumplen regex DNI, edad >= 18 y email)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono, refCuenta)
VALUES ('12345678A', 'Ana', 'García', 'ana@email.com', 25, 'Calle Mayor 1', 600111222,
ListaRefsCuentas());

INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono, refCuenta)
VALUES ('87654321B', 'Luis', 'Pérez', 'luis@email.com', 40, 'Calle Menor 2', 600333444,
ListaRefsCuentas());

-- Insertar Cuentas usando constructores tipados
-- Cuenta Corriente para Ana (apuntando a la oficina 1)
INSERT INTO Cuenta VALUES (
    CorrienteUdt('ES9100001111222233334444', 1111222233334444, SYSDATE, 0,
ListaRefsClientes(),
    (SELECT REF(o) FROM Oficina o WHERE o.Codigo = 1)
);

-- Cuenta de Ahorro para Luis (con 2.5% de interés)
INSERT INTO Cuenta VALUES (
    AhorroUdt('ES9100005555666677778888', 5555666677778888, SYSDATE, 0,
ListaRefsClientes(), 2.5)
);

-- Actualizar la relación bidireccional (Añadir clientes a cuentas y cuentas a clientes)
UPDATE Cuenta c
SET c.refCliente = c.refCliente MULTISET UNION ListaRefsClientes((SELECT REF(c1) FROM
Cliente c1 WHERE c1.Dni = '12345678A'))
WHERE c.Iban = 'ES9100001111222233334444';

UPDATE Cliente c1

```

```

SET cl.refCuenta = cl.refCuenta MULTISET UNION ListaRefsCuentas((SELECT REF(c) FROM
Cuenta c WHERE c.Iban = 'ES910000111122233334444'))
WHERE cl.Dni = '12345678A';

COMMIT;
PROMPT Datos base insertados correctamente.

PROMPT =====;
PROMPT 2. PRUEBAS DE RESTRICCIONES DE INTEGRIDAD (ERRORES ORA-);
PROMPT =====;

PROMPT > Prueba 2.1: DNI con formato erroneo (Falla por CHECK cli_dni_chk)
INSERT INTO Cliente (Dni, Nombre, Email, Apellidos, Edad, Direccion, Telefono, refCuenta)
VALUES ('123A45678', 'Falso', 'falso@email.com', 'Dni', 30, 'Dir', 600000000,
ListaRefsCuentas());

PROMPT > Prueba 2.2: Cliente menor de edad (Falla por CHECK cli_edad_val)
INSERT INTO Cliente (Dni, Nombre, Email, Apellidos, Edad, Direccion, Telefono, refCuenta)
VALUES ('11111111C', 'Joven', 'joven@email.com', 'Menor', 16, 'Dir', 600000000,
ListaRefsCuentas());

PROMPT > Prueba 2.3: Operacion con fecha anterior a creacion de la cuenta (Falla por
Trigger TRG_VAL_FECHAS_OP)
INSERT INTO Operacion VALUES (
    EfectivoUdt(999, 'Retirada en pasado', SYSDATE - 10, 50,
        (SELECT REF(c) FROM Cuenta c WHERE c.Iban = 'ES910000111122233334444'),
        (SELECT REF(o) FROM Oficina o WHERE o.Codigo = 1)
    )
);

PROMPT =====;
PROMPT 3. PRUEBAS DE OPERACIONES Y TRIGGERS DE SALDO (DEBEN FUNCIONAR);
PROMPT =====;

PROMPT > 3.1: Hacemos un Ingreso de 1000 euros en la cuenta de Ana (Operacion en
Efectivo)
-- *Nota: En diseño objeto, un ingreso suma (como un pago negativo o un trigger adaptado)
-- Asumimos que el trigger de actualizacion lo maneja como resta del origen. Para simular
ingreso:
UPDATE Cuenta c SET c.Saldo_actual = 1000 WHERE Iban = 'ES910000111122233334444';
PROMPT Ingreso manual inicializado para pruebas de extraccion.

PROMPT > 3.2: Hacemos una Retirada de 200 euros en la cuenta de Ana (Operacion en
Efectivo)
INSERT INTO Operacion VALUES (
    EfectivoUdt(1, 'Cajero', SYSDATE, 200,
        (SELECT REF(c) FROM Cuenta c WHERE c.Iban = 'ES910000111122233334444'),
        (SELECT REF(o) FROM Oficina o WHERE o.Codigo = 1)
    )
);

PROMPT > 3.3: Ana hace una transferencia de 300 euros a Luis
INSERT INTO Operacion VALUES (
    TransferenciaUdt(2, 'Regalo', SYSDATE, 300,
        (SELECT REF(c) FROM Cuenta c WHERE c.Iban = 'ES910000111122233334444'),
        (SELECT REF(c) FROM Cuenta c WHERE c.Iban = 'ES9100005555666677778888')
    )
);

```

```

);

PROMPT > ESTADO DE SALDOS ESPERADO:
PROMPT > Ana empezo con 1000 - 200 - 300 = 500 euros.
PROMPT > Luis empezo con 0 + 300 transferidos = 300 euros.
SELECT Iban, Saldo_actual FROM Cuenta;

PROMPT =====;
PROMPT 4. PRUEBA DE DISPARADOR DE FONDOS INSUFICIENTES (CHECK SALDO);
PROMPT =====;

PROMPT > Ana intenta sacar 600 euros pero solo tiene 500.
PROMPT > El trigger intentara restar el saldo y el CHECK (Saldo_actual >= 0) de la tabla
Cuenta lo impedira.
INSERT INTO Operacion VALUES (
    EfectivoUdt(3, 'Compra TV', SYSDATE, 600,
        (SELECT REF(c) FROM Cuenta c WHERE c.Iban = 'ES9100001111222233334444'),
        (SELECT REF(o) FROM Oficina o WHERE o.Codigo = 1)
    )
);

PROMPT > Comprobamos que el saldo de Ana sigue intacto en 500 euros
SELECT Iban, Saldo_actual FROM Cuenta WHERE Iban = 'ES9100001111222233334444';

PROMPT =====;
PROMPT 5. PRUEBA DEL PROCEDIMIENTO DE INTERESES NOCTURNOS;
PROMPT =====;

PROMPT > Ejecutamos el pago de intereses manual (Liquidacion de la cuenta de ahorro de
Luis al 2.5%)
EXEC PROC_INTERESES_NOCTURNOS;

PROMPT > Comprobamos que el saldo de Luis ha subido (Tenia 300, y se le ha sumado su
interes)
SELECT Iban, Saldo_actual FROM Cuenta WHERE Iban = 'ES9100005555666677778888';

COMMIT;
PROMPT PRUEBAS FINALIZADAS.

```

13.3.5. bd-bancaria-or-pruebas-postgresql.sql

```

-- =====
-- Script: bd-bancaria-or-pruebas-postgresql.sql
-- SGBD: PostgreSQL
-- Modelo: Objeto-relacional
-- Proposito: Insercion de datos y pruebas funcionales del
--           modelo objeto-relacional en PostgreSQL
-- Requiere: bd-bancaria-or-postgresql.sql
-- Observaciones:
--   - Inserta datos base en oficinas, clientes y cuentas.
--   - Utiliza arrays para representar referencias entre objetos.
--   - Ejecuta pruebas negativas de restricciones y triggers
--     (algunos errores SQL son esperados).
--   - Verifica el funcionamiento de los triggers de operaciones
--     y del procedimiento de calculo de intereses.
-- =====

```

```

\echo '=====
\echo '0. LIMPIEZA DE DATOS PREVIOS'
\echo '=====

-- TRUNCATE CASCADE vacía las tablas y todas las que heredan de ellas automáticamente
TRUNCATE TABLE Operacion CASCADE;
TRUNCATE TABLE Cuenta CASCADE;
TRUNCATE TABLE Cliente CASCADE;
TRUNCATE TABLE Oficina CASCADE;

\echo 'Datos anteriores eliminados correctamente.'

\echo '=====
\echo '1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE)'
\echo '=====

-- Insertar Oficinas (Inicializando arrays vacíos)
INSERT INTO Oficina (Codigo, Telefono, Direccion, refCorriente, refEfectivo)
VALUES (1, 976111222, 'Paseo Independencia, Zaragoza', '{}', '{}');

INSERT INTO Oficina (Codigo, Telefono, Direccion, refCorriente, refEfectivo)
VALUES (2, 911222333, 'Gran Vía, Madrid', '{}', '{}');

-- Insertar Clientes
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono, refCuenta)
VALUES ('12345678A', 'Ana', 'García', 'ana@email.com', 25, 'Calle Mayor 1', 600111222,
'{}');

INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono, refCuenta)
VALUES ('87654321B', 'Luis', 'Pérez', 'luis@email.com', 40, 'Calle Menor 2', 600333444,
'{}');

-- Insertar Cuentas directamente en las TABLAS HIJAS (Gracias a INHERITS)
-- Cuenta Corriente para Ana
INSERT INTO CuentaCorriente (Iban, Numero, Fecha_creacion, Saldo_actual, refCliente,
refOficina)
VALUES ('ES9100001111222233334444', 1111222233334444, NOW(), 0, ARRAY['12345678A'], 1);

-- Cuenta de Ahorro para Luis (con 2.5% de interés)
INSERT INTO CuentaAhorro (Iban, Numero, Fecha_creacion, Saldo_actual, refCliente,
Tipo_interes)
VALUES ('ES9100005555666677778888', 5555666677778888, NOW(), 0, ARRAY['87654321B'], 2.5);

-- Actualizar la relación bidireccional (Añadir cuentas al array de los clientes)
UPDATE Cliente SET refCuenta = array_append(refCuenta, 'ES9100001111222233334444') WHERE
Dni = '12345678A';
UPDATE Cliente SET refCuenta = array_append(refCuenta, 'ES9100005555666677778888') WHERE
Dni = '87654321B';

\echo 'Datos base insertados correctamente.'

\echo '=====
\echo '2. PRUEBAS DE RESTRICCIONES DE INTEGRIDAD (ERRORES ESPERADOS)'
\echo '=====

\echo '> Prueba 2.1: Email con formato erróneo (Falla por CHECK de la tabla)'
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono, refCuenta)

```



```

VALUES ('123A45678', 'Falso', 'Dni', 'falsoemail.com', 30, 'Dir', 600000000, '{}');

\echo '> Prueba 2.2: Cliente menor de edad (Falla por CHECK de la tabla)'
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono, refCuenta)
VALUES ('11111111C', 'Joven', 'Menor', 'joven@email.com', 16, 'Dir', 600000000, '{}');

\echo '> Prueba 2.3: Operación con fecha anterior a creación de cuenta (Falla por Trigger
PL/pgSQL)'
INSERT INTO OperacionEfectivo (Codigo, Concepto, Fecha, Cantidad, refCuentaOrigen,
refOficina)
VALUES (999, 'Retirada en pasado', NOW() - INTERVAL '10 days', 50,
'ES910000111122233334444', 1);

\echo '=====
\echo '3. PRUEBAS DE OPERACIONES Y TRIGGERS DE SALDO'
\echo '=====

\echo '> 3.1: Inicializamos el saldo de Ana a 1000 euros para pruebas'
UPDATE CuentaCorriente SET Saldo_actual = 1000 WHERE Iban = 'ES910000111122233334444';

\echo '> 3.2: Hacemos una Retirada de 200 euros en la cuenta de Ana (OperacionEfectivo)'
INSERT INTO OperacionEfectivo (Codigo, Concepto, Fecha, Cantidad, refCuentaOrigen,
refOficina)
VALUES (1, 'Cajero', NOW(), 200, 'ES910000111122233334444', 1);

\echo '> 3.3: Ana hace una transferencia de 300 euros a Luis (Transferencia)'
INSERT INTO Transferencia (Codigo, Concepto, Fecha, Cantidad, refCuentaOrigen,
refCuentaDestino)
VALUES (2, 'Regalo', NOW(), 300, 'ES910000111122233334444', 'ES9100005555666677778888');

\echo '> ESTADO DE SALDOS ESPERADO:'
\echo '> Ana: 1000 - 200 - 300 = 500 euros.'
\echo '> Luis: 0 + 300 transferidos = 300 euros.'
-- Nota: Hacemos SELECT sobre "Cuenta" (tabla padre) y gracias a INHERITS mostrará los
saldos de Corriente y Ahorro
SELECT Iban, Saldo_actual FROM Cuenta;

\echo '=====
\echo '4. PRUEBA DE DISPARADOR DE FONDOS INSUFICIENTES (CHECK SALDO)'
\echo '=====

\echo '> Ana intenta sacar 600 euros pero solo tiene 500. El CHECK (Saldo_actual >= 0) lo
impedirá.'
INSERT INTO OperacionEfectivo (Codigo, Concepto, Fecha, Cantidad, refCuentaOrigen,
refOficina)
VALUES (3, 'Compra TV', NOW(), 600, 'ES910000111122233334444', 1);

\echo '> Comprobamos que el saldo de Ana sigue intacto en 500 euros'
SELECT Iban, Saldo_actual FROM Cuenta WHERE Iban = 'ES910000111122233334444';

\echo '=====
\echo '5. PRUEBA DEL PROCEDIMIENTO DE INTERESES NOCTURNOS'
\echo '=====

\echo '> Ejecutamos el pago de intereses (Liquidación de la cuenta de ahorro de Luis al
2.5%)'
CALL proc_intereses_nocturnos();

```

```
\echo '> Comprobamos que el saldo de Luis ha subido (Tenía 300, se le suma el 2.5% =
307.50)'
SELECT Iban, Saldo_actual FROM Cuenta WHERE Iban = 'ES9100005555666677778888';

\echo 'PRUEBAS FINALIZADAS.
```

13.3.6. bd-bancaria-or-pruebas-db2.sql

```
-- =====
-- Script: bd-bancaria-or-pruebas-db2.sql
-- SGBD: IBM DB2
-- Modelo: Objeto-relacional
-- Proposito: Insercion de datos y pruebas funcionales del
--           modelo objeto-relacional en DB2
-- Requiere: bd-bancaria-or-db2.sql
-- Observaciones:
--   - El script se ejecuta sobre la base de datos PRACTDB2.
--   - Utiliza el terminador '@' para las sentencias SQL.
--   - Inserta datos base en oficinas, clientes y cuentas.
--   - Incluye pruebas negativas de restricciones
--     (algunos errores SQL0545N son esperados).
--   - Simula operaciones bancarias y verifica los saldos finales.
-- =====

-- Conexion a la base de datos donde se ejecutaran las pruebas
CONNECT TO DB2INST1
@

-- =====
-- 0. LIMPIEZA DE DATOS (Borrando desde La raíz de La jerarquía)
-- =====

DELETE FROM Operacion
@

DELETE FROM Cuenta
@

DELETE FROM Cliente
@

DELETE FROM Oficina
@

COMMIT
@

-- =====
-- 1. INSERCIÓN DE DATOS BASE
-- =====

INSERT INTO Oficina (oid_oficina,Codigo, Telefono, Direccion)
VALUES (OficinaUdt('1'), 1, 976111222, 'Paseo Independencia, Zaragoza')
@

INSERT INTO Oficina (oid_oficina, Codigo, Telefono, Direccion)
```

```

VALUES (OficinaUdt('2'), 2, 911222333, 'Gran Vía, Madrid')
@

INSERT INTO Cliente (oid_cliente, Dni, Nombre, Email, Apellidos, Edad, Direccion,
Telefono)
VALUES (ClienteUdt('1'), '12345678A', 'Ana', 'ana@email.com', 'García', 25, 'Calle Mayor
1', 600111222)
@

INSERT INTO Cliente (oid_cliente, Dni, Nombre, Email, Apellidos, Edad, Direccion,
Telefono)
VALUES (ClienteUdt('2'), '87654321B', 'Luis', 'luis@email.com', 'Pérez', 40, 'Calle Menor
2', 600333444)
@

INSERT INTO CuentaCorriente (oid_cuenta, Iban, Numero, Fecha_creacion, Saldo_actual,
refOficina)
VALUES (CorrienteUdt('1'), 'ES9100001111222233334444', 1111222233334444, CURRENT DATE, 0,
1)
@

INSERT INTO CuentaAhorro (oid_cuenta, Iban, Numero, Fecha_creacion, Saldo_actual,
Tipo_interes)
VALUES (AhorroUdt('2'), 'ES9100005555666677778888', 5555666677778888, CURRENT DATE, 0,
2.5)
@

COMMIT
@

-- =====
-- 2. PRUEBAS DE RESTRICCIONES
-- =====

-- Cliente menor de edad (debe fallar)
INSERT INTO Cliente (oid_cliente, Dni, Nombre, Apellidos, Email, Edad, Direccion,
Telefono)
VALUES (ClienteUdt('3'),
'11111111C', 'Joven', 'Menor', 'joven@email.com', 16, 'Dir', 600000000)
@

-- Saldo negativo (debe fallar)
INSERT INTO Cuenta (oid_cuenta, Iban, Numero, Fecha_creacion, Saldo_actual)
VALUES (CuentaUdt('3'), 'ES0000000000000000000000', 1111, CURRENT DATE, -100)
@

-- =====
-- 3. OPERACIONES
-- =====

UPDATE Cuenta
SET Saldo_actual = 1000
WHERE Iban = 'ES9100001111222233334444'
@

INSERT INTO OperacionEfectivo (oid_operacion, Codigo, Concepto, Fecha, Cantidad,
refCuentaOrigen, refOficina)

```

```

VALUES (EfectivoUdt('1'), 1, 'Cajero', CURRENT_TIMESTAMP, 200,
'ES9100001111222233334444', 1)
@

INSERT INTO Transferencia (oid_operacion, Codigo, Concepto, Fecha, Cantidad,
refCuentaOrigen, refCuentaDestino)
VALUES (TransferenciaUdt('2'), 2, 'Regalo', CURRENT_TIMESTAMP, 300,
'ES9100001111222233334444', 'ES9100005555666677778888')
@

UPDATE Cuenta
SET Saldo_actual = Saldo_actual - 200 - 300
WHERE Iban = 'ES9100001111222233334444'
@

UPDATE Cuenta
SET Saldo_actual = Saldo_actual + 300
WHERE Iban = 'ES9100005555666677778888'
@

-- =====
-- 4. FONDOS INSUFICIENTES
-- =====

UPDATE Cuenta
SET Saldo_actual = Saldo_actual - 600
WHERE Iban = 'ES9100001111222233334444'
@

-- =====
-- 5. ESTADO FINAL
-- =====

SELECT Iban, Saldo_actual FROM Cuenta
@

CONNECT RESET
@

```

13.4. Anexo IV. Implementación orientada a objetos (db4o)

El código fuente completo de la implementación orientada a objetos se incluye en el fichero proyecto-db4o.zip. Este fichero contiene las clases Java del modelo, la clase principal de ejecución y las librerías necesarias para ejecutar la aplicación.